



UNIVERSITÀ DEGLI STUDI DI SALERNO

Dipartimento di Informatica

Corso di Laurea Triennale in Informatica

TESI DI LAUREA

Applicazione web per stimare la sicurezza delle Applets IFTTT

RELATORE

Prof. Fabio Palomba

Dott. Giammaria Giordano

Università degli Studi di Salerno

CANDIDATO

Michele Pesce

Matricola: 0512115152

Anno Accademico 2023-2024

Questa tesi è stata realizzata nel

sesa^{lab}
SOFTWARE ENGINEERING
SALERNO

"L'intelligenza artificiale è un generatore di bozze, non di testi definitivi. Abbiamo ancora bisogno della mente dell'uomo per controllare che non ci siano errori, falsità, incongruenze."

Abstract

L'automazione dei processi tramite Applets IFTTT è spesso adottata senza considerare i potenziali rischi di vulnerabilità, che possono portare a danni fisici, digitali o personali. In questo lavoro, ho sviluppato un'applicazione web con l'obiettivo di aumentare la consapevolezza degli utenti riguardo i rischi associati all'attivazione di applet potenzialmente dannose. L'applicazione non solo classifica le applet come sicure o non sicure, ma fornisce anche informazioni specifiche sui possibili problemi di sicurezza. La ricerca ha portato alla creazione di uno strumento utile e affidabile per affrontare diverse sfaccettature dei problemi legati all'automazione, offrendo un supporto concreto agli utenti nella gestione della sicurezza digitale.

Indice

Elenco delle Figure	iv
Elenco delle Tabelle	v
1 Introduzione	1
1.1 Contesto applicativo	1
1.2 Motivazioni e Obiettivi	3
1.3 Risultati ottenuti	4
1.4 Struttura della Tesi	5
2 Background e stato dell'arte	8
2.1 IFTTT	8
2.1.1 Applet	12
2.2 Problemi di sicurezza: Creatori di Applet Malintenzionati	13
2.2.1 Analisi dei Flussi di Dati	14
2.2.2 Strumenti per Mitigare i Rischi	16
2.2.3 Estensione del Lavoro	16
2.3 Problemi di Sicurezza: Applet Intrinsecamente Dannose	16
2.3.1 Reazioni a catena: i rischi per la sicurezza delle catene di applet	18
2.3.2 Estensione del lavoro e proposte di miglioramento	19
2.4 Percezione dell'utente dei rischi associati alle applet	19

2.4.1	Integrazioni e possibili sviluppi	20
2.5	Preoccupazioni degli utenti in relazione all'uso delle applet	21
2.5.1	Integrazioni e sviluppi futuri	22
2.6	Rischi legati all'uso delle applet IFTTT da parte degli utenti reali . . .	23
2.6.1	Contributo del mio lavoro	23
3	Metodo di ricerca	25
3.1	Analisi dei dati e Creazione del dataset	26
3.1.1	Analisi del dataset etichettato	26
3.1.2	Analisi del dataset non etichettato	28
3.1.3	Calcolo della soglia di similarità	31
3.1.4	Creazione del nuovo dataset	33
3.1.5	Analisi e ingegnerizzazione del nuovo dataset	35
3.2	Processo di Addestramento del Modello	37
3.2.1	Applicazione del metodo empirico	41
3.2.2	Algoritmi scelti e fasi conclusive	43
3.3	Ingegneria di IFTTT Security Extension	43
3.3.1	Analisi	44
3.3.2	Progettazione di alto livello	45
3.3.3	Progettazione di basso livello	49
3.4	Modulo LinkLoader	52
3.5	Modulo DatabaseLoader	53
3.6	Integrazione del modello	54
3.6.1	Modalità di estrazione del modello	54
3.6.2	Gestione delle predizioni	56
4	Analisi dei Risultati	59
4.1	Discussione sul dataset	60
4.2	Discussione sul modello di Machine Learning	61
4.3	Risultati del Web Scraping	63
4.4	Valutazione di IFTTT Security Extension	64
4.4.1	Valutazione delle funzionalità	65
4.4.2	Valutazione dell'usabilità e dell'efficienza	67

5 Conclusioni	69
5.1 Sviluppi futuri	70
Bibliografia	72

Elenco delle figure

1.1	Esempio di applet IFTTT	2
2.1	Distribuzione delle violazioni di segretezza e integrità nelle applet analizzate [1].	17
3.1	Boxplot affiancati che confrontano il numero di installazioni delle applet, differenziate in base al fatto che siano state create dal proprietario del servizio o meno.	30
3.2	Conteggio totale per categoria di azione.	31
3.3	Numero di installazioni per applet in base al numero di servizi connessi.	32
3.4	Distribuzione delle applet per etichetta	35
3.5	Distribuzione delle applet per etichette dopo il bilanciamento	38
3.6	Storyboard IFTTT Security Extension	46
3.7	Modello E/R di IFTTT Security Extension	48
3.8	Modello E/R di IFTTT Security Extension	50
3.9	Modello E/R di IFTTT Security Extension	51
4.1	Homepage di IFTTT Security Extension	65
4.2	Predizione di IFTTT Security Extension	66
4.3	Navbar di IFTTT Security Extension	68

Elenco delle tabelle

3.1	Tabella dei requisiti funzionali	58
4.1	Risultati delle metriche di valutazione del modello	63
4.2	Matrice di confusione del modello	63

CAPITOLO 1

Introduzione

1.1 Contesto applicativo

Automatizzare processi significa ottenere risultati con il minimo sforzo. Un esempio semplice è una **funzione informatica**: con una serie di input si generano output pronti per un prossimo utilizzo. Una funzione può essere richiamata in un'altra funzione, che utilizza il suo output per produrre un risultato più complesso e articolato.

Lo scopo di IFTTT¹. è simile: combinare due o più "funzioni" in un processo generale che produce un'**automazione comoda e veloce**, chiamata **Applet** (o ricetta). Un'Applet può essere paragonata alla creazione di una torta: per prepararla, è necessario aggiungere gli ingredienti nell'ordine corretto e procedere con la cottura. Questi due passaggi, sebbene separati, sono strettamente correlati: la cottura sarà possibile solo dopo che gli ingredienti sono stati aggiunti.

Nella terminologia di IFTTT, queste operazioni sono chiamate **Trigger** e **Action**. All'attivazione del Trigger (un **evento specifico**), segue l'esecuzione di un'azione definita dall'Action.

¹IFTTT (If This Then That) è una piattaforma che consente di creare automazioni tra diversi servizi e dispositivi digitali.

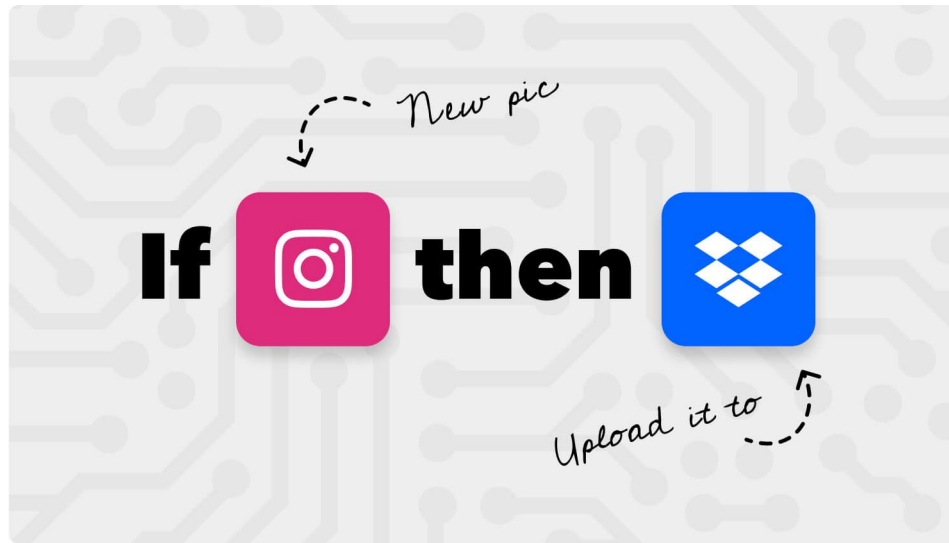


Figura 1.1: Esempio di applet IFTTT

Come mostra l'immagine, se viene caricata una nuova foto su Instagram, questa verrà salvata su Dropbox². In questo caso, l'Applet coinvolge un **social network**, ma spesso possono coinvolgere **dispositivi IoT**³, **applicazioni di storage**, **strumenti mobile**, ecc..

Consideriamo l'Applet: *"Se la temperatura di casa supera una soglia di X gradi, allora apri la finestra."*. Questa automazione coinvolge dispositivi IoT in una Smart Home, offrendo vantaggi in termini di comodità, soprattutto quando la casa è vuota. Tuttavia, tale comodità potrebbe distogliere l'attenzione da aspetti importanti. Ad esempio, un ladro, conoscendo questa automazione, potrebbe sfruttarla per entrare in casa e causare un **danno fisico**.

Un altro scenario potrebbe riguardare un utente che, ogni volta che svolge un'attività fisica, pubblica automaticamente i risultati su Facebook. Se l'utente dovesse svolgere un'attività di recupero a seguito di una malattia, tali informazioni potrebbero essere pubblicate, causando imbarazzo (**danno personale**).

²Dropbox è un servizio di cloud storage che permette di salvare e condividere file online.

³Gli IoT (Internet of Things) sono dispositivi connessi a Internet che possono comunicare e interagire tra loro e con altri sistemi.

Per gestire comodamente gli allegati di posta elettronica, un utente potrebbe creare un'Applet che carica tutti gli allegati delle email appena ricevute su OneDrive. Tuttavia, se l'utente ricevesse un'email con un allegato dannoso, e utilizzasse OneDrive per sincronizzare più dispositivi, l'allegato dannoso verrebbe automaticamente copiato su più dispositivi, aumentando la probabilità che venga eseguito per errore, provocando un **danno di sicurezza informatica**.

1.2 Motivazioni e Obiettivi

Uno studio condotto dalla **Carnegie Mellon University**[1] ha mostrato che, su un campione di 19.323 applet, circa il 50% presenta potenziali rischi per la segretezza o una violazione dell'integrità personale (o entrambi).

Questi dati evidenziano la necessità di prestare **maggiore attenzione** nell'attivazione di tali applet e suggeriscono l'importanza di sviluppare uno strumento in grado di prevedere quando un'applet potrebbe risultare dannosa. Il mio obiettivo, infatti, è contribuire alla creazione di strumenti che possano **allertare** gli utenti IFTTT prima di attivare un'automazione potenzialmente pericolosa.

La scelta di sviluppare un'applicazione web risponde all'esigenza di raggiungere un pubblico più ampio, in quanto IFTTT è molto usato tramite **browser web**[2]. Come accennato, l'applicazione non si limiterà a riconoscere le applet dannose, ma fornirà anche spiegazioni dettagliate sul motivo per cui un'applet potrebbe comportare rischi.

Infine, si osserva una **scarsità di strumenti adeguati** per affrontare questo problema. Per sviluppare un modello di machine learning⁴ efficace è essenziale disporre di un dataset affidabile. Tuttavia, i dataset⁵ disponibili online raramente si concentrano sulla sicurezza delle applet e, quando presenti, sono spesso di dimensioni insufficienti per garantire un accurato addestramento del modello.

⁴Un modello di machine learning è un sistema computazionale che impara a partire dai dati per effettuare previsioni o prendere decisioni.

⁵Un dataset è una raccolta di dati, tipicamente organizzata in una tabella o in un formato strutturato, che può essere utilizzata per analisi, modellazione statistica e apprendimento automatico.

1.3 Risultati ottenuti

Come già accennato, attualmente non esistono strumenti adeguati per risolvere il problema delle applet dannose. L'applicazione web sviluppata si basa su un modello di machine learning progettato per stimare se un'automazione sia dannosa e, in tal caso, quale tipo di danno potrebbe causare. Questo modello è stato addestrato utilizzando un dataset con target classificato *da 0 a 3*, dove ciascun valore indica il livello di **danno potenziale**.

Il modello è stato addestrato mediante un **approccio empirico**, che ha previsto la sperimentazione di diverse **combinazioni di parametri** per ciascun algoritmo. Questa metodologia ha permesso di identificare l'algoritmo più efficace, portando a un'accuratezza del 95% durante la fase di valutazione e validazione.

La web app è stata sviluppata con un forte focus sull'usabilità e sulla manutenibilità. Per raggiungere questi obiettivi, è stata adottata un'architettura client/server, separando il **frontend** dall'interfaccia utente e il **backend** dalla logica applicativa e dall'integrazione con il modello di intelligenza artificiale. Il backend è implementato utilizzando **Spring Boot** [3], un framework Java, e gestisce la logica applicativa e le interazioni con il database. Inoltre, è stata integrata una funzione per restituire il valore stimato dal modello, esportato come file PMML[4] e utilizzato nell'ambiente Java.

Per migliorare la manutenibilità, il backend è stato strutturato seguendo il pattern MVC (Model-View-Controller)[5], suddividendo le funzionalità in:

- **Controller:** Gestisce le richieste HTTP su endpoint⁶ specifici.
- **Service:** Implementa la logica applicativa per i requisiti funzionali.
- **Repository:** Gestisce l'interazione con il database.

Il frontend è stato sviluppato con **React**[6], un framework noto per la creazione di interfacce web **reattive** e robuste. La progettazione dell'interfaccia si è concentrata su un numero limitato di pagine ricche di contenuti, per garantire un'esperienza utente ottimale.

⁶Un endpoint è un punto finale di comunicazione in una rete o in un'API, attraverso il quale si possono inviare e ricevere dati.

Infine, per creare il database necessario, è stata utilizzata la tecnica del **web scraping**⁷. Poiché non erano disponibili API IFTTT o risorse esterne adeguate, è stato creato un processo di scraping per raccogliere dati direttamente da pagine web specifiche. Questo ha permesso di acquisire 9070 applet per il test della web app e circa 1000 servizi abilitati alla piattaforma.

1.4 Struttura della Tesi

La presente tesi è organizzata in **quattro capitoli** principali, ognuno dei quali esplora un aspetto fondamentale del lavoro svolto, contribuendo a una comprensione completa e integrata del progetto. Di seguito viene fornita una panoramica dei capitoli e dei loro contenuti.

Capitolo 2: Background e stato dell'arte

Il secondo capitolo delinea il contesto teorico e le conoscenze pregresse su cui si basa il progetto. Inizia con una panoramica dettagliata del sistema IFTTT e delle sue applet (Sezione 2.1), fornendo una base di comprensione fondamentale per il lettore. Successivamente, si analizzano i problemi di sicurezza associati alle applet, inclusi i rischi posti da creatori malintenzionati e le strategie per mitigarli (Sezioni 2.2 e 2.3). Questo capitolo esplora anche la percezione degli utenti riguardo ai rischi e le loro preoccupazioni (Sezioni 2.4 e 2.5), offrendo un quadro complessivo di come gli utenti interagiscono con le applet e quali problematiche emergono. Infine, si discute il rischio legato all'uso reale delle applet, sottolineando il contributo specifico di questa ricerca nel contesto delle preoccupazioni individuate (Sezione 2.6).

Capitolo 3: Metodo di ricerca

Il terzo capitolo descrive il metodo di ricerca utilizzato per affrontare le problematiche identificate. Inizia con un'analisi approfondita dei dati e la creazione del dataset necessario (Sezione 3.1), che include sia il dataset etichettato che quello non

⁷Il web scraping è il processo di estrazione automatizzata di dati da siti web utilizzando strumenti e script per raccogliere informazioni strutturate da pagine web.

etichettato. Questa fase è cruciale poiché fornisce le basi su cui verranno effettuate le analisi successive. La sezione 3.2 illustra il processo di addestramento del modello, spiegando le tecniche e gli algoritmi utilizzati per ottenere risultati significativi. La Sezione 3.3 si concentra sull'ingegneria di IFTTT Security Extension, descrivendo come è stata progettata e implementata la web app per affrontare le problematiche di sicurezza. Inoltre, vengono discussi i moduli specifici, come LinkLoader e Database-Loader, e la loro integrazione nel sistema complessivo (Sezioni 3.4 e 3.5). La Sezione 3.6 analizza l'integrazione del modello nel contesto dell'intero progetto, garantendo che le soluzioni proposte siano coerenti e funzionanti.

Capitolo 4: Analisi dei Risultati

Il quarto capitolo è dedicato all'analisi dei risultati ottenuti attraverso le metodologie e le tecniche implementate. Questo capitolo fornisce una valutazione critica dei risultati e analizza l'efficacia delle soluzioni adottate.

La **Sezione 4.1** si concentra sulla discussione del dataset utilizzato. Questa sezione esamina la qualità dei dati e l'efficacia del dataset nell'affrontare le problematiche di sicurezza delle applet, fornendo una panoramica sulle caratteristiche e sulle sfide incontrate durante la raccolta e la preparazione dei dati.

La **Sezione 4.2** è dedicata alla valutazione del modello di Machine Learning applicato. In questa sezione, vengono presentate le performance del modello, analizzando i risultati ottenuti rispetto agli obiettivi prefissati e confrontando le prestazioni con i modelli esistenti. Questo permette di comprendere l'impatto delle tecniche di Machine Learning nel migliorare la sicurezza delle applet.

La **Sezione 4.3** analizza i risultati del web scraping. Viene valutata l'efficacia delle tecniche di estrazione dei dati, discutendo i risultati ottenuti e l'affidabilità delle informazioni raccolte. Questa sezione è cruciale per comprendere come le informazioni ottenute attraverso il web scraping contribuiscano alla sicurezza e alla funzionalità del sistema.

Infine, la **Sezione 4.4** fornisce una valutazione complessiva dell'IFTTT Security Extension. Questa sezione discute come il modello e le tecniche implementate abbiano

influito sulla sicurezza delle applet, analizzando i benefici apportati e le aree di miglioramento.

Capitolo 5: Conclusioni

Nel capitolo delle conclusioni si riassumono i principali contributi della ricerca, che ha portato allo sviluppo di diversi strumenti utili per l'analisi e la sicurezza delle applet su IFTTT. Si discutono anche i limiti della ricerca, come l'approfondimento della soglia di similarità e l'integrazione futura di trigger e action per migliorare ulteriormente il database. Infine, vengono suggeriti sviluppi futuri legati all'analisi delle percezioni degli utenti, alla creazione di applet personalizzate e a metodi innovativi di sensibilizzazione, come l'uso della realtà aumentata e della visione artificiale.

Questa struttura è progettata per guidare il lettore attraverso una narrazione coerente e logica del progetto, dalla definizione del contesto e delle problematiche alla presentazione dei risultati e delle conclusioni, assicurando una

Background e stato dell'arte

Il campo dei problemi di sicurezza relativi a IFTTT è relativamente **nuovo e poco esplorato**, ma alcuni studi hanno fornito spunti utili per comprendere il contesto e le sfide che questa piattaforma presenta. Alcune ricerche si concentrano sui rischi intrinseci delle applet, mentre altre analizzano la sicurezza dal punto di vista dei creatori di applet malintenzionati.

Prima di addentrarsi nei dettagli delle ricerche e dei problemi di sicurezza, è fondamentale comprendere meglio **cos'è IFTTT e come funziona**.

2.1 IFTTT

IFTTT, acronimo di *"If This Then That"* è una piattaforma online che consente agli utenti di creare automazioni tra diversi servizi e dispositivi digitali. L'obiettivo principale di IFTTT è semplificare le attività quotidiane **automatizzando azioni** che altrimenti richiederebbero tempo e attenzione se eseguite manualmente. Gli utenti possono creare automazioni chiamate **applets** che collegano due o più servizi o dispositivi. Ogni applet funziona tramite una regola condizione-azione, dove un **trigger** (condizione) attiva un **action** (azione).

Le automazioni di IFTTT possono essere categorizzate in diverse aree principali, che includono:

1. Social Network

Questa categoria comprende piattaforme di **social media** che permettono di automatizzare attività come postare aggiornamenti o salvare contenuti. Esempi di servizi includono:

- (a) **Facebook**: Pubblicare post, aggiornare la foto del profilo, salvare foto in Google Drive.
- (b) **Twitter**: Postare tweet, monitorare hashtag, aggiungere tweet preferiti a una lista.
- (c) **Instagram**: Salvare foto in cloud storage, condividere foto su altri social media.

2. Dispositivi IoT (Internet of Things)

Questa categoria include **dispositivi smart** controllabili tramite IFTTT, molto usati per la gestione della casa intelligente. Esempi includono:

- (a) **Philips Hue**: Accendere o spegnere le luci, cambiare il colore in base a condizioni specifiche.
- (b) **Nest Thermostat**: Regolare la temperatura, attivare modalità particolari basate su posizione o ora del giorno.
- (c) **Smart Lock**: Bloccare o sbloccare le porte, ricevere notifiche quando la porta è aperta.

3. Servizi di Cloud Storage

Questa categoria riguarda servizi per **salvare e gestire file online**, come documenti e foto. Esempi includono:

- (a) **Google Drive**: Salvare allegati email, organizzare file in cartelle specifiche.
- (b) **Dropbox**: Salvare foto e video da dispositivi mobili, creare backup automatici.

- (c) **OneDrive:** Sincronizzare file tra dispositivi, organizzare documenti secondo criteri specifici.

4. Email e Comunicazione

Servizi che automatizzano attività legate alla gestione delle email e alla comunicazione. Esempi includono:

- (a) **Gmail:** Inviare notifiche per email da contatti specifici, archiviare automaticamente email in cartelle.
- (b) **Slack:** Inviare messaggi a canali quando si verifica un evento, come un nuovo follower su Twitter.
- (c) **Microsoft Outlook:** Creare regole per l'invio automatico di email o per l'organizzazione della posta.

5. Domotica e Sicurezza

Servizi per la sicurezza domestica e l'automazione delle funzioni casalinghe. Esempi includono:

- (a) **Ring Doorbell:** Ricevere notifiche quando qualcuno suona il campanello, attivare luci o sirene.
- (b) **Arlo:** Registrare video su cloud storage quando viene rilevato movimento.
- (c) **Wyze:** Attivare la registrazione video al rilevamento di suoni, inviare notifiche a dispositivi mobili.

6. Produttività e Pianificazione

Strumenti per migliorare l'efficienza e l'organizzazione. Esempi includono:

- (a) **Google Calendar:** Aggiungere eventi automaticamente, ricevere promemoria.
- (b) **Todoist:** Creare task automatici basati su email ricevute, aggiornare liste di cose da fare.
- (c) **Evernote:** Salvare note, organizzare informazioni automaticamente.

7. Notizie e Informazioni

Servizi per aggiornamenti e informazioni su vari argomenti. Esempi includono:

- (a) **RSS Feed:** Ricevere aggiornamenti da siti web, salvare articoli in Pocket o Evernote.
- (b) **Google News:** Inviare notifiche su notizie di interesse.
- (c) **Weather:** Ricevere previsioni meteo giornaliere o avvisi per condizioni avverse.

8. Intrattenimento

Servizi relativi a media e divertimento. Esempi includono:

- (a) **Spotify:** Aggiungere canzoni a playlist in base a condizioni specifiche.
- (b) **YouTube:** Ricevere notifiche per nuovi video da canali seguiti.
- (c) **Netflix:** Salvare automaticamente titoli in una lista quando vengono aggiunti nuovi episodi.

9. Fitness e Salute

Servizi per monitorare e migliorare la salute e il benessere. Esempi includono:

- (a) **Fitbit:** Sincronizzare dati di fitness con altri servizi, come Google Sheets o MyFitnessPal.
- (b) **Strava:** Monitorare attività sportive e salvare risultati in cloud storage.
- (c) **Apple Health:** Integrare dati sanitari con altre applicazioni o servizi.

La web app di IFTTT permette di accedere a questi servizi e attivare applet preimpostate da sviluppatori, dal canale stesso o da utenti casuali. Gli utenti possono anche creare le proprie automazioni, collegando servizi di vari canali attraverso regole condizione-azione, per personalizzare le proprie esperienze di automazione.

2.1.1 Applet

Come descritto precedentemente, le azioni automatizzate su servizi e dispositivi sono gestite tramite automazioni chiamate Applet. Un'applet è composta da due elementi principali:

- **Trigger:** È l'evento o condizione che avvia l'applet. Rappresenta la parte "If This" della regola. Ad esempio, un trigger può essere l'ora del giorno, un nuovo post su un social network, la ricezione di un'email, o un cambiamento di stato in un dispositivo smart (come l'accensione di una luce).
- **Action:** È l'azione che viene eseguita automaticamente quando il Trigger si verifica. Rappresenta la parte "Then That" della regola. Può includere attività come inviare una notifica, salvare un file, pubblicare un post, accendere una luce o aggiornare un database.

In sintesi, un **Trigger** innesca un **Action**, creando così un'automazione basata su una condizione specifica.

Un'applet può anche essere arricchita con i seguenti elementi:

- **Code Filter:** Consente di personalizzare ulteriormente il comportamento delle applet aggiungendo condizioni aggiuntive. Ad esempio, un'applet che riceve una notifica ogni volta che un nuovo tweet contiene una determinata parola chiave può utilizzare un code filter per filtrare i tweet in base a parametri come la lingua, l'autore del tweet o la presenza di hashtag specifici. Il code filter rappresenta un sottoinsieme di eventi scatenanti di un certo trigger.
- **Query:** Si riferisce a richieste di informazioni o dati utilizzati per interagire con i servizi e i dispositivi supportati dalla piattaforma. Le query possono essere utilizzate per recuperare dati, come le condizioni meteorologiche attuali, e utilizzarli all'interno di un'applet per eseguire azioni specifiche. Ad esempio, un'applet potrebbe includere una query per recuperare la temperatura o la previsione del tempo e utilizzarle come trigger per attivare l'applet o come input per un'azione dell'applet.

- **Ingredienti:** Sono variabili dinamiche che personalizzano l'azione di un'applet in base ai dati forniti dal trigger. Ad esempio, un'applet che riceve una notifica ogni volta che piove nella tua città potrebbe utilizzare ingredienti come la temperatura e l'intensità della pioggia per personalizzare il messaggio della notifica. Gli ingredienti aiutano a rendere le azioni più specifiche e rilevanti.

2.2 Problemi di sicurezza: Creatori di Applet Malintenzionati

Le applet IFTTT spesso richiedono l'autorizzazione degli utenti per accedere a dati sensibili o per avere il consenso all'accesso ai vari canali. Questa autorizzazione riguarda sia il trigger che l'azione, poiché coinvolgono servizi distinti.

L'uso di dati sensibili può rappresentare un rischio significativo quando applet malintenzionate sono in gioco. Ecco come potrebbe avvenire un attacco[7]:

- **Creazione dell'applet dannosa:** Un utente malintenzionato crea un'applet che sembra utile, ad esempio, un'applet per monitorare i tweet su un argomento specifico.
- **Manipolazione del code filter:** L'attaccante inserisce codice malevolo nel code filter dell'applet, che cattura e invia le credenziali dell'utente a un server controllato dall'attaccante.
- **Pubblicazione dell'applet:** L'applet viene pubblicata e resa disponibile agli altri utenti di IFTTT.
- **Esecuzione dell'applet da parte dell'utente vittima:** Un utente ignaro attiva l'applet. Quando il trigger si verifica, il codice malevolo viene eseguito e le credenziali vengono rubate. Sebbene il code filter non possa accedere direttamente alle credenziali, può manipolare i dati provenienti dal trigger per raccogliere informazioni sensibili.
- **Utilizzo delle credenziali rubate:** Le credenziali ottenute possono essere utilizzate per accedere agli account dell'utente e compiere ulteriori attività dannose.

I creatori malintenzionati possono effettuare tre tipi di attacchi[7]:

1. **Privacy:** La privacy può essere compromessa attraverso due principali classi di attacchi basati su URL:
 - **Attacco tramite caricamento di URL:** Se i canali richiedono un URL pubblico per caricare media, un malintenzionato può inserire un URL pubblico con dati sensibili e inviarli a un server malevolo.
 - **Attacco al markup dell'URL:** Utilizzando un tag `<img>` per nascondere parametri URL, un malintenzionato può esfiltrare dati sensibili quando un utente clicca sull'immagine.
2. **Integrità:** L'integrità dei dati può essere compromessa se un code filter altera i dati del trigger e dell'azione, rendendo i dati non affidabili.
3. **Disponibilità:** Gli attacchi alla disponibilità possono verificarsi quando le azioni sono ignorate tramite comandi di skip nel code filter, riducendo la disponibilità dei dati o delle funzionalità.

Le applets possono essere soggette ad **attacchi di forza bruta**. IFTTT abbrevia automaticamente gli URL in URL con il formato `"http://ift.tt/"` nel markup generato per gli utenti. Questo significa che questa base di URL è fissa, quindi un utente malintenzionato può scrivere un URL completo (completamente a caso) e azzeccare l'URL generato automaticamente da IFTTT contenente delle informazioni dell'utilizzatore.

2.2.1 Analisi dei Flussi di Dati

Uno studio recente ha esplorato il problema dei **code filters compromessi** nelle applet IFTTT, rivelando che tali flussi di dati possono compromettere seriamente la sicurezza degli utenti [7]. Per valutare i rischi, i ricercatori hanno creato due **dataset**, uno per i trigger e uno per le azioni delle applet, ognuno etichettato per indicare potenziali problemi di sicurezza.

Etichettatura dei Trigger

I trigger sono stati classificati con le seguenti etichette:

- **Privati:** contengono informazioni sensibili.
- **Pubblici:** accessibili pubblicamente senza restrizioni.
- **Disponibili:** dati disponibili pubblicamente, ma rilevanti per l'utente.

Etichettatura delle Azioni

Le azioni sono state invece etichettate come:

- **Pubblico:** dati che possono essere condivisi apertamente.
- **Non attendibile:** dati che possono essere manipolati da fonti esterne.
- **Disponibile:** azioni che, se ignorate, possono influire sull'utente.

Violazioni Identificate

In base a questa classificazione, lo studio ha individuato tre principali violazioni dei flussi di dati:

1. **Violazioni della privacy:** quando dati da trigger privati sono inviati a azioni pubbliche.
2. **Violazioni dell'integrità:** quando dati provenienti da qualsiasi trigger sono inviati a un'azione non attendibile.
3. **Violazioni della disponibilità:** quando trigger disponibili causano azioni non eseguite correttamente.

L'analisi ha rivelato che *circa il 30%* delle applet esaminate sono vulnerabili a violazioni della privacy, con oltre *8 milioni di utenti* esposti. Inoltre, *il 98%* delle applet è risultato vulnerabile a violazioni dell'integrità, coinvolgendo più di *18 milioni di utenti*. Per quanto riguarda le violazioni della disponibilità, queste rappresentano lo *0,5%* delle applet analizzate.

2.2.2 Strumenti per Mitigare i Rischi

Per affrontare questi problemi, i ricercatori hanno sviluppato un monitor chiamato **FlowIT**, basato su **JSFlow** [8], un sistema per il monitoraggio dinamico del flusso di informazioni in JavaScript. FlowIT permette di tracciare il flusso dei dati all'interno delle applet, identificando potenziali rischi legati alla privacy, all'integrità e alla disponibilità delle informazioni.

2.2.3 Estensione del Lavoro

Sebbene lo studio sui code filters sia un passo importante verso la comprensione delle vulnerabilità nelle applet IFTTT, esistono ulteriori aspetti da considerare. Il mio lavoro non si limita ai code filters, ma si concentra su una più ampia gamma di scenari di rischio, includendo quelli in cui i trigger richiedono accesso a dati privati per il corretto funzionamento dell'applet. Inoltre, il mio approccio si focalizza sulla consapevolezza dell'utente: una **interfaccia web** è stata progettata per avvisare gli utenti quando condividono informazioni sensibili tramite applet potenzialmente vulnerabili a **attacchi di cybersecurity**, contribuendo a una protezione più completa e proattiva.

2.3 Problemi di Sicurezza: Applet Intrinsecamente Dannose

Un'altra categoria di problemi di sicurezza riguarda le applet che, pur non contenendo code filter malevoli, possono comunque causare danni fisici o digitali. Queste applet possono avere trigger e azioni che portano a danni esterni, come descritto nel paragrafo 1.1.

Esistono due tipi principali di violazioni delle politiche sul flusso di informazioni[1]:

1. **Segretezza:** Le violazioni della segretezza si verificano quando informazioni riservate diventano accessibili a un pubblico più vasto. Ad esempio, un'applet che pubblica automaticamente su social media senza il consenso dell'utente può violare la segretezza delle informazioni private.

2. **Violazioni dell'integrità:** Si verificano quando informazioni provenienti da fonti meno attendibili influenzano quelle di fonti più affidabili. Ad esempio, un sensore di movimento difettoso potrebbe attivare erroneamente un'applet, causando l'accensione delle luci anche in assenza di una reale presenza nel giardino.

Un modo per analizzare i rischi di tali applet è l'applicazione di etichette di *segretezza* e *integrità*, come proposto nella ricerca condotta dalla Carnegie Mellon University [1].

- Le **etichette di segretezza** indicano chi può accedere ai dati associati a un evento.
- Le **etichette di integrità** indicano chi può innescare un'azione.

Per valutare il rischio, si utilizzano due reticoli, uno per la segretezza e uno per l'integrità, che categorizzano i trigger e le action in base al loro livello di accessibilità e controllo. Secondo i risultati della ricerca, **circa il 49,9%** delle applet analizzate presenta potenziali violazioni di segretezza o integrità [1]. Più nel dettaglio, il 22,9% delle applet viola l'integrità, il 16,7% la segretezza, mentre un ulteriore 10,3% presenta violazioni in entrambe le categorie.

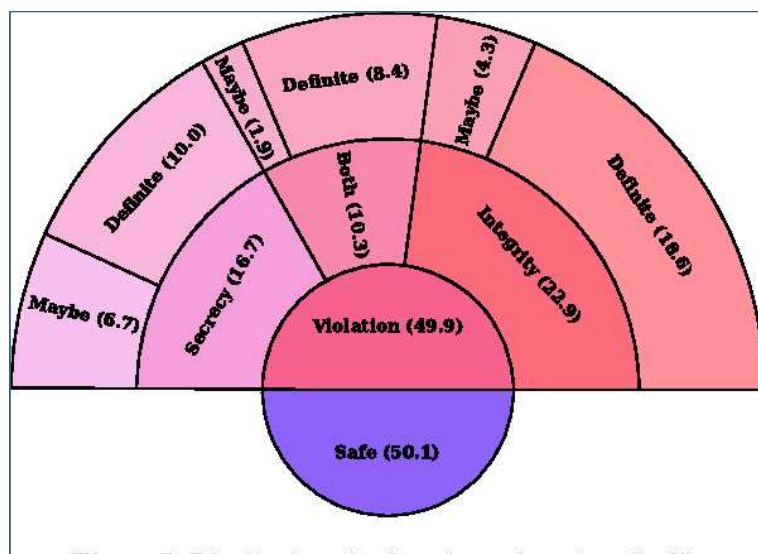


Figura 2.1: Distribuzione delle violazioni di segretezza e integrità nelle applet analizzate [1].

Un aspetto interessante emerso dalla ricerca è che, sebbene il 16% delle violazioni di segretezza comporti una condivisione pubblica delle informazioni, l'84% riguarda la condivisione con gruppi ristretti o limitati, suggerendo che gli utenti spesso sottovalutano i rischi di divulgazione in questi contesti.

2.3.1 Reazioni a catena: i rischi per la sicurezza delle catene di applet

Come accennato in precedenza, l'esecuzione di un applet ne può attivare un'altra, creando così situazioni indesiderate. Ci sono 2 tipologie di catene[1]:

- **Collegamenti diretti:** Le ricette A e B sono direttamente collegate se il canale di azione di A e il canale di trigger di B sono gli stessi e l'azione di A attiva il trigger di B. Ad esempio, l'azione "Invia una email al mio account email" attiverà direttamente il trigger "Nuova email ricevuta" se entrambi provengono dal canale Gmail dell'utente. In breve, se la ricetta A termina con un'azione, l'azione di A farà partire a catena la ricetta B, dove l'azione corrisponderà all'evento che scatena il trigger di B.
- **Collegamenti fisici:** Due ricette possono essere collegate fisicamente se l'azione della prima ricetta influisce su un mezzo fisico o canale come temperatura, suono o luce, e il trigger della seconda ricetta viene attivato dai cambiamenti dello stesso mezzo o canale. In questo caso, il concetto è simile a quello delle ricette collegate direttamente, ma interviene anche il mezzo fisico.

Le connessioni tra le ricette attraverso il mezzo fisico sono spesso sottili. Ad esempio, spegnere una ventola tramite una presa intelligente farà aumentare la temperatura in una stanza, il che potrebbe attivare una ricetta il cui trigger è la temperatura che sale al di sopra di una certa soglia. Per tracciare le connessioni fisiche, si etichettano i trigger e le action rilevanti con un canale fisico (ad esempio, temperatura, suono o luce) e un evento fisico (ad esempio, cambio di livello sopra, accendere). Gli eventi fisici ricadono approssimativamente in due categorie: quelli che modificano il livello di un mezzo fisico e quelli che controllano un cambiamento di livello, ad esempio, accendere un riscaldamento e un termostato Nest che verifica

se la temperatura è sopra una soglia (c'è un evento fisico dovuto all'accensione dei riscaldamenti, quindi questi controllano il cambiamento di livello).

2.3.2 Estensione del lavoro e proposte di miglioramento

Il mio lavoro si propone di estendere queste analisi, affrontando non solo i rischi legati alla segretezza e all'integrità, ma anche altri aspetti della sicurezza, come i potenziali danni fisici o gli attacchi informatici derivanti dall'uso inappropriato delle applet. Un'altra estensione importante riguarda la **consapevolezza dell'utente**. La mancanza di percezione del rischio è un fattore chiave nei casi di utilizzo di applet insicure. Il mio progetto prevede la creazione di una **interfaccia web** che informi proattivamente gli utenti sui potenziali pericoli delle applet che intendono attivare, fornendo loro strumenti per valutare meglio i rischi legati all'uso di IFTTT.

Questa nuova direzione non solo cerca di analizzare le applet in modo più completo, ma si concentra anche sull'educazione e sulla protezione dell'utente finale, contribuendo a mitigare i rischi non riconosciuti nella ricerca precedente.

2.4 Percezione dell'utente dei rischi associati alle applet

Le applet basate su regole *Event-Condition-Action* (ECA), come quelle offerte dalla piattaforma IFTTT, sollevano preoccupazioni in merito alla sicurezza e alla privacy degli utenti. Queste **preoccupazioni** derivano dalla natura delle regole ECA, che automatizzano le azioni sulla base di eventi predefiniti. Un esempio può essere l'attivazione di un'applicazione che invia un messaggio ogni volta che viene ricevuta una notifica o il salvataggio automatico di un file in un servizio cloud. Tuttavia, l'automazione di tali processi comporta potenziali rischi, specialmente quando si tratta di applet progettate da terze parti che richiedono accesso a dati sensibili.

Uno degli aspetti chiave della ricerca sui rischi legati alle applet riguarda la capacità di individuare e classificare automaticamente le regole ECA in base ai rischi per la sicurezza e la privacy. In uno studio[9], i ricercatori hanno applicato tecniche di **Natural Language Processing (NLP)** per analizzare semanticamente i trigger, le azioni e le descrizioni delle applet, proponendo un modello basato su **Bidirectional Encoder**

Representations from Transformers (BERT) per la classificazione dei rischi [10]. Il modello ha raggiunto un'accuratezza di classificazione pari all'88%, evidenziando un'alta capacità di identificare potenziali minacce per gli utenti.

Il problema della percezione dell'utente sui rischi associati alle applet emerge come elemento cruciale per comprendere il comportamento degli utenti stessi. A tal fine, lo studio ha indagato se gli utenti fossero in grado di riconoscere i rischi classificati dal modello e se questi influenzassero la loro decisione di attivare o meno un'applet. Le domande chiave a cui gli utenti hanno risposto sono state:

- Gli utenti percepiscono il rischio individuato dal modello come reale?
- I rischi percepiti influenzano la loro decisione di attivare l'applet?

I risultati indicano che la maggior parte degli utenti ha riconosciuto le classificazioni del modello come allineate alle loro percezioni, mostrando una corrispondenza tra l'identificazione automatica dei rischi e la valutazione soggettiva degli utenti. Inoltre, l'evidenza dei rischi ha avuto un impatto significativo sul comportamento degli utenti, inducendoli a evitare l'attivazione di applet percepite come non sicure. Questo dimostra che una corretta classificazione e segnalazione dei rischi può fungere da strumento efficace per migliorare la sicurezza delle applet in ambito IFTTT.

Nonostante la precisione del modello di classificazione, vi sono state situazioni in cui alcune applet innocue sono state identificate come potenzialmente dannose, suggerendo la necessità di un'analisi più approfondita e di criteri di classificazione più articolati. Alcuni utenti hanno sottolineato che la valutazione del rischio non può basarsi esclusivamente su un approccio automatico, ma deve essere supportata da un **quadro decisionale più ampio**.

2.4.1 Integrazioni e possibili sviluppi

Il mio lavoro si inserisce in questo contesto, cercando di migliorare ulteriormente la capacità di individuare e classificare i rischi per la sicurezza e la privacy delle applet. Basandomi su un dataset pubblico fornito dai ricercatori, ho sviluppato un nuovo modello di **machine learning** che utilizza algoritmi di classificazione e regressione più avanzati. L'obiettivo è quello di ottenere una precisione ancora maggiore nella

valutazione dei rischi, riducendo al minimo i falsi positivi e i falsi negativi, e di fornire agli utenti un sistema di allerta più completo e accurato.

Le ulteriori integrazioni proposte includono lo sviluppo di un'interfaccia utente che segnali i rischi in tempo reale, offrendo una spiegazione dettagliata del perché un'applet potrebbe essere considerata pericolosa. Questo approccio combinerebbe le tecniche di machine learning con un feedback continuo da parte degli utenti, migliorando così l'affidabilità del sistema e la sicurezza complessiva delle applet.

2.5 Preoccupazioni degli utenti in relazione all'uso delle applet

La crescente diffusione di dispositivi smart home e Internet of Things (IoT) ha portato all'adozione massiccia di applet automatizzate, come quelle offerte da IFTTT, che collegano diversi dispositivi e servizi per facilitare la vita quotidiana. Tuttavia, l'automazione porta con sé anche preoccupazioni riguardanti la sicurezza e la privacy, soprattutto quando gli utenti sono meno consapevoli dei potenziali rischi associati all'attivazione delle applet.

Uno degli aspetti chiave nello studio delle interazioni tra utenti e applet è comprendere quanto gli utenti finali siano in grado di valutare i rischi potenziali derivanti dall'uso di applet in vari contesti. Ad esempio, è importante capire se gli utenti sono in grado di identificare le implicazioni di sicurezza quando attivano un'applet che accede a dati personali o esegue azioni in dispositivi domestici connessi. Uno studio ha indagato la percezione degli utenti riguardo a queste preoccupazioni [11], cercando di rispondere a domande fondamentali come:

- Quanto gli utenti si preoccupano dei rischi associati all'uso di applet IFTTT basate su una semplice descrizione?
- Come i fattori contestuali influenzano la valutazione del rischio degli utenti?

Per rispondere a queste domande, è stato condotto un sondaggio coinvolgendo un ampio numero di partecipanti, che hanno espresso le loro preoccupazioni riguardo all'uso di applet IFTTT popolari, con particolare attenzione all'ambiente

IoT. I risultati hanno evidenziato che la maggior parte degli utenti **non percepiva** un rischio immediato o elevato leggendo una semplice descrizione di un'applet. Infatti, solo una piccola percentuale (12,99%) si è dichiarata "**estremamente preoccupata**", mentre la maggior parte ha dichiarato di **non essere preoccupata affatto** (37,42%).

Tuttavia, quando sono stati presi in considerazione fattori **contestuali** specifici, come la posizione del dispositivo IoT o il momento dell'esecuzione dell'applet, è emerso che tali dettagli hanno influenzato significativamente la percezione del rischio da parte degli utenti. Ogni partecipante ha mostrato un **aumento** delle preoccupazioni almeno per uno dei fattori contestuali proposti, dimostrando come un maggiore dettaglio possa modificare la valutazione di sicurezza.

2.5.1 Integrazioni e sviluppi futuri

Questo studio [11] si è concentrato principalmente su applet legate all'ambiente IoT, ma i rischi per la sicurezza e la privacy possono estendersi a una vasta gamma di applet, anche al di fuori del contesto IoT. Il mio lavoro mira a espandere questa analisi, offrendo un supporto più ampio per identificare rischi potenziali non solo nelle applet IoT, ma anche in altre categorie, come quelle che coinvolgono servizi cloud, social media e dispositivi mobili.

Per raggiungere questo obiettivo, la **web app** non solo segnalerà i rischi per la sicurezza e la privacy, ma fornirà anche una valutazione del rischio in tempo reale, tenendo conto dei fattori contestuali specifici per ogni tipo di applet. Questo strumento ha lo scopo di aumentare la consapevolezza degli utenti, offrendo informazioni chiare sui rischi associati alle applet in modo che possano prendere decisioni più informate. Un sistema di monitoraggio continuo e personalizzato migliorerà ulteriormente la protezione contro i rischi di sicurezza, estendendo la portata rispetto a studi precedenti.

2.6 Rischi legati all'uso delle applet IFTTT da parte degli utenti reali

Diversi studi hanno analizzato i rischi associati all'uso di applet condivise pubblicamente, ma pochi si sono concentrati sull'impatto effettivo di queste problematiche per gli **utenti reali**, cioè coloro che creano e utilizzano le regole IFTTT nella vita quotidiana. Un'indagine su utenti reali ha evidenziato che una percentuale significativa delle regole create dagli utenti potrebbe presentare vulnerabilità legate alla segretezza e all'integrità [12]. In particolare:

- Circa il 57% delle regole IFTTT create dai partecipanti ha evidenziato potenziali violazioni della segretezza o dell'integrità.
- Tuttavia, solo una piccola percentuale di queste regole (10%) comportava un reale rischio di perdita di dati sensibili.
- Poco meno del 20% delle regole esaminate poneva rischi significativi per la coerenza o la correttezza delle informazioni.

Questo suggerisce che, sebbene molte applet possano tecnicamente comportare violazioni, tali rischi risultano raramente gravi nella pratica quotidiana. Gli utenti spesso non percepiscono le loro applet come potenzialmente dannose, anche se esprimono preoccupazioni generali riguardo alla sicurezza e alla privacy.

Un altro aspetto rilevante emerso da tali analisi riguarda i rischi di **sorveglianza accidentale**. Le regole IFTTT possono infatti catturare involontariamente attività di altre persone oltre all'utente che ha creato l'applet, specialmente quando si tratta di dispositivi come videocamere di sorveglianza o altri sensori ambientali. Questo tipo di rischio non era stato precedentemente considerato, ma si dimostra particolarmente importante in contesti dove i dispositivi IoT monitorano l'ambiente domestico e possono violare la privacy di persone "incidental".

2.6.1 Contributo del mio lavoro

Il mio lavoro si propone di estendere ulteriormente l'analisi sui rischi derivanti dall'uso delle applet IFTTT, includendo una gamma più ampia di possibili minacce.

Oltre ai rischi di segretezza e integrità già trattati, la mia ricerca si concentra anche su:

- **Rischi fisici:** ad esempio, le conseguenze di malfunzionamenti o attacchi a dispositivi come serrature smart o sistemi di allarme, che possono compromettere la sicurezza fisica degli utenti.
- **Rischi informatici:** includendo potenziali attacchi ai dati personali memorizzati o trasmessi attraverso le applet.
- **Ampliamento dei contesti:** analizzare le applet non solo in ambito IoT, ma anche in altre categorie (ad esempio, social media, servizi cloud), dove regole mal configurate possono esporre dati sensibili o causare danni imprevisti.

CAPITOLO 3

Metodo di ricerca

In questo capitolo, verrà descritto l'intero processo di creazione di **IFTTT Security Extension**, a partire dall'analisi dei dati disponibili, passando per l'ampliamento del dataset utilizzato per addestrare un **classificatore**, fino alla progettazione e **popolazione** del database contenente le applet attualmente disponibili su IFTTT, concludendo con la **progettazione e implementazione** dell'applicazione web.

Il percorso intrapreso si basa sulla necessità di rispondere a diverse domande chiave, utili a guidare ogni fase del lavoro:

- **(RQ1)** *Quali criteri devono essere considerati per determinare se un'applet sia potenzialmente dannosa?*
- **(RQ2)** *Perché è importante ampliare il dataset esistente?*
- **(RQ3)** *Qual è lo scopo principale della creazione di una web app per la valutazione delle applet?*
- **(RQ4)** *In che modo una web app può aiutare gli utenti a identificare e comprendere i rischi legati alle applet IFTTT?*
- **(RQ5)** *Come vengono recuperate e gestite le applet disponibili su IFTTT per la loro integrazione nel database?*

3.1 Analisi dei dati e Creazione del dataset

I dati pubblici su applet classificate come dannose o non dannose sono estremamente rari. Nonostante ciò, ho preferito condurre approfondite ricerche alla ricerca di un dataset già esistente, piuttosto che classificare manualmente un campione di applet sulla base della loro sicurezza.

L'unico dataset disponibile che ho trovato è stato quello sviluppato in [10], ma presentava alcune limitazioni rilevanti (RQ2) :

- Il numero di record era limitato.
- Le applet non dannose erano state rimosse.

Il mio obiettivo era costruire un dataset più ampio e diversificato. Per farlo, ho combinato questo dataset con un altro contenente oltre 50.000 tuple e ho applicato una tecnica di **apprendimento semi-supervisionato**, ispirata al lavoro descritto in [10], per classificare automaticamente le applet del nuovo dataset utilizzando i record del dataset già etichettato.

Per gestire l'assenza di applet non dannose, ho adottato un approccio basato sulla **similarità testuale**. Poiché entrambi i dataset erano composti principalmente da descrizioni in linguaggio naturale, ho applicato un'analisi della similarità del testo: se due applet risultavano sufficientemente diverse, sotto una certa soglia di similarità, classificavo l'applet come non dannosa. Questo metodo ha funzionato in modo analogo a un algoritmo di clustering¹, poiché ha permesso di organizzare le applet in gruppi di applet potenzialmente dannose e non dannose in maniera automatizzata.

Infine dopo aver creato il nuovo dataset, sono passato a una fase di **analisi** per cercare di coglierne la qualità e i possibili problemi.

3.1.1 Analisi del dataset etichettato

Come anticipato, il dataset etichettato utilizzato in questo lavoro contiene un totale di 497 record, ognuno dei quali rappresenta informazioni dettagliate sulle

¹Un algoritmo di clustering è una tecnica di machine learning che raggruppa automaticamente dati simili in insiemi distinti senza supervisione.

applet e include un campo chiave chiamato *target*, che valuta la sicurezza delle applet.

Il campo *target* assume valori compresi tra 1 e 3, ognuno dei quali riflette un problema di sicurezza specifico, come descritto in [10][1] (RQ1):

- **1 - Danni personali:** applet che possono causare la perdita o compromissione di dati sensibili, legati principalmente al comportamento dell'utente. Queste applet possono esporre informazioni personali senza che l'utente ne sia consapevole.
- **2 - Danni fisici:** applet che possono comportare danni fisici o materiali, dove il danno è esterno e inflitto da terze parti. Queste applet potrebbero, ad esempio, controllare dispositivi IoT che impattano la sicurezza fisica.
- **3 - Danno di sicurezza informatica:** applet che possono compromettere la sicurezza dei servizi online o distribuire malware. Questi casi includono interruzioni di servizi o attacchi informatici che sfruttano le vulnerabilità delle applet.

Oltre al campo *target*, il dataset contiene altri campi significativi che forniscono ulteriori dettagli sulle applet, tra cui:

- **triggerTitle:** descrive l'evento che attiva l'applet. Questo campo indica il tipo di evento, come una specifica data o azione.
- **triggerChannelTitle:** identifica il canale o servizio associato all'evento scatenante. Ad esempio, il canale potrebbe essere "Date & Time" o "Location".
- **actionTitle:** specifica l'azione che l'applet esegue una volta attivata, come la pubblicazione di un messaggio o l'attivazione di un dispositivo IoT.
- **actionChannelTitle:** indica il canale o servizio che esegue l'azione. Potrebbe trattarsi di piattaforme come "Facebook Pages" o "Smart Home Devices".
- **title:** fornisce una breve descrizione dell'applet, in genere un riassunto conciso della sua funzionalità.
- **desc:** offre una descrizione più dettagliata del comportamento dell'applet, spiegando come si comporta in un determinato contesto.

- **justification:** giustifica il motivo per cui l'applet viene considerata rischiosa, fornendo un contesto sugli effetti potenziali.

Nel corso dell'analisi, non è stato necessario effettuare un'approfondita pulizia dei dati o bilanciamento, poiché il dataset si presenta già come pulito e ben strutturato, come indicato in [10]. Questo ha permesso di concentrarsi principalmente sulla valutazione della sicurezza e sui metodi di classificazione utilizzati per distinguere le applet dannose da quelle sicure.

3.1.2 Analisi del dataset non etichettato

Il dataset analizzato, trovato su Kaggle ed esaminato da *STEVEN YU*, contiene un totale di 50.228 applet e le seguenti informazioni:

- **byServiceOwner:** Indica se l'applet è stata creata dal proprietario del servizio (`True`) o da un utente esterno (`False`).
- **channels:** Elenco dei canali o servizi utilizzati dall'applet, con dettagli come il nome, il logo, il colore del marchio e la disponibilità.
- **description:** Fornisce una descrizione testuale dell'applet, spiegando brevemente la sua funzione o scopo.
- **friendly_id:** Identificativo amichevole dell'applet, usato per rappresentarla in modo leggibile.
- **id:** Identificativo univoco dell'applet nel sistema IFTTT.
- **installs_count:** Numero di installazioni dell'applet.
- **name:** Nome dell'applet, che riflette il suo scopo o funzione.
- **permissions:** Elenco dei permessi richiesti dall'applet (attualmente vuoto).
- **pro_features:** Indica se l'applet richiede funzioni premium (`True` o `False`).
- **requires_android_app:** Indica se l'applet richiede l'app Android (`True` o `False`).

- **requires_ios_app**: Indica se l'applet richiede l'app iOS (`True` o `False`).
- **requires_mobile_app**: Indica se l'applet richiede l'app mobile (iOS o Android) (`True` o `False`).
- **service_names**: Lista dei nomi dei servizi utilizzati dall'applet, come Google Calendar o Office 365 Calendar.
- **service_slug**: Identificatore univoco del servizio utilizzato dall'applet, ad esempio `office_365_calendar`.
- **services**: Elenco dei servizi associati all'applet, simile a `channels`, ma con dettagli più tecnici.
- **speed**: Indica la velocità con cui l'applet viene eseguita (ad esempio, `Polling Applets usually run within 1 hour`).
- **uniq_permissions**: Elenco di permessi unici richiesti dall'applet (attualmente vuoto).
- **services_len**: Numero di servizi utilizzati dall'applet.
- **service_triggers**: Descrive i trigger utilizzati dall'applet (ad esempio, un evento su Google Calendar).
- **service_actions**: Descrive le azioni eseguite dall'applet (ad esempio, aggiungere un evento al Google Calendar).
- **triggers_category**: Categoria del trigger dell'applet (ad esempio, `Calendars & scheduling`).
- **actions_category**: Categoria dell'azione dell'applet (ad esempio, `Not Found`).

La prima analisi effettuata riguarda la **distribuzione** del numero di installazioni delle applet in base alla variabile *byServiceOwner*.

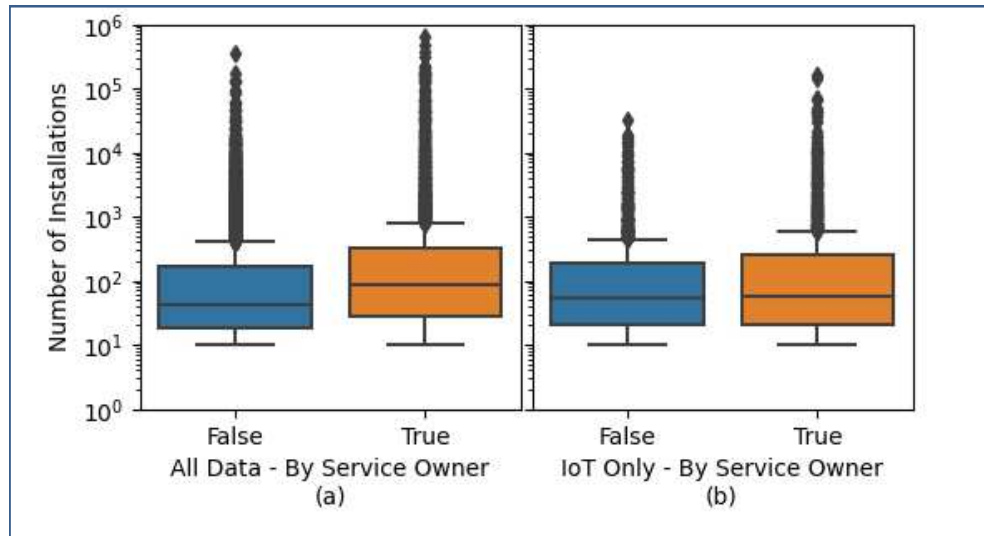


Figura 3.1: Boxplot affiancati che confrontano il numero di installazioni delle applet, differenziate in base al fatto che siano state create dal proprietario del servizio o meno.

Nel grafico 3.1 si osservano due **boxplot**² affiancati, che confrontano il numero di installazioni delle applet. Il primo boxplot mostra i dati per tutte le applet, mentre il secondo mostra solo quelle relative all'IoT.

Si può concludere che, sia per le applet che coinvolgono canali IoT sia per le restanti, il numero di installazioni è **equivalente** per le applet create dal proprietario del servizio e per quelle create da terzi.

Nel grafico 3.2 è mostrato il conteggio totale delle installazioni per ciascuna **categoria di azione**, ordinato in modo decrescente. Si osserva che la maggior parte delle azioni è legata ai **dispositivi IoT**. Questo risultato è confermato dall'immagine 3.1, dove le due distribuzioni risultano molto simili.

Questa ipotesi è ulteriormente confermata dai risultati mostrati nell'immagine 3.3. Infatti il grafico a barre nella figura mostra il numero di installazioni per applet in base al numero di servizi connessi, suddividendo i dati in due categorie: dati generali e solo IoT. Le barre rappresentano la somma totale delle installazioni per ciascun numero di servizi connessi:

- **Asse X:** Rappresenta il numero di servizi connessi a ciascuna applet.

²Un boxplot è un grafico che visualizza la distribuzione dei dati tramite cinque misure statistiche principali: il minimo, il primo quartile, la mediana, il terzo quartile e il massimo, evidenziando anche i valori anomali.

Action	
Lighting	1249927.0
Environment control & monitoring	211910.0
Smart hubs & systems	182405.0
Security & monitoring systems	142853.0
Appliances	66230.0
Gardening	48414.0
Power monitoring & management	31851.0
Health & fitness	24858.0
DIY electronics	7690.0
Blinds	6946.0
Routers & computer accessories	6155.0
Television & cable	1295.0
dtype:	float64

Figura 3.2: Conteggio totale per categoria di azione.

- **Asse Y:** Mostra il numero totale di installazioni, utilizzando una scala logaritmica per rendere visibili anche le piccole differenze tra i numeri.
- **Barre Blu:** Rappresentano il numero di installazioni per le applet generali, che includono tutti i tipi di servizi.
- **Barre Arancioni:** Indicano il numero di installazioni per le applet IoT, che si collegano specificamente ai servizi IoT.

Per ulteriori analisi e dettagli sul dataset, si può consultare il notebook dell'autore.

3.1.3 Calcolo della soglia di similarità

Come descritto in 3.1, uno dei principali problemi del dataset etichettato riguarda l'**assegnazione** delle applet come innocue. In base alla ricerca di [10], un'applet considerata innocua viene etichettata con il valore 0. Ho utilizzato questa informazione per integrare le applet attraverso un processo **semi-automatico**, come spiegato nel paragrafo 3.1.4.

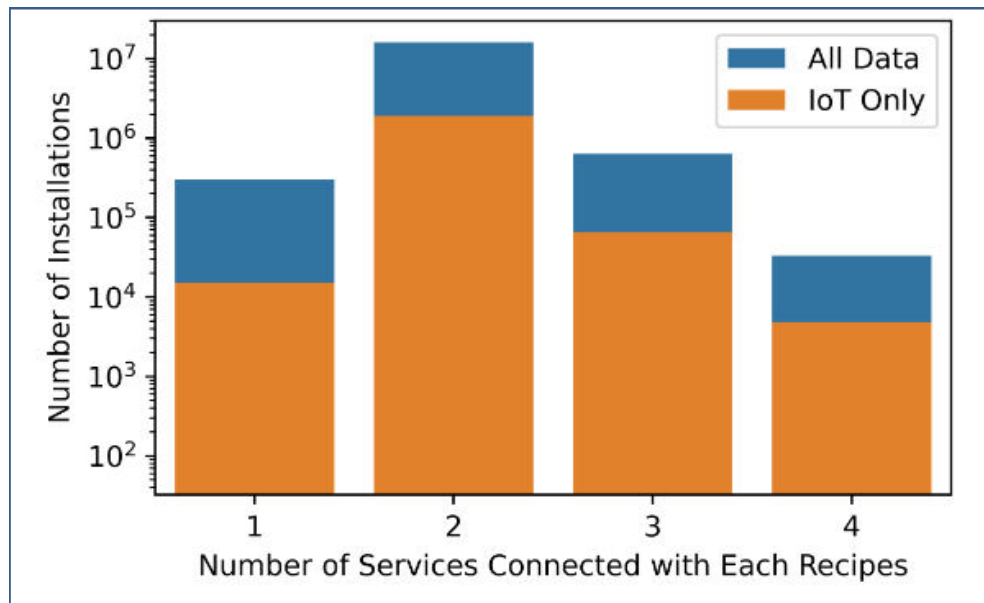


Figura 3.3: Numero di installazioni per applet in base al numero di servizi connessi.

Per distinguere tra applet dannose e non dannose, mi baso sulla **similarità del testo**. È essenziale definire una **soglia** di similarità che consenta di classificare correttamente un'applet come non dannosa, evitando falsi positivi, ovvero applet dannose erroneamente classificate come non dannose.

Nel seguito di questo paragrafo, descrivo il processo utilizzato per determinare la soglia più appropriata:

```

1  # Inizializzo un array per le massime similarità
2  # Numero di righe di X_unlabeled
3  max_sim = np.zeros(X_unlabeled.shape[0])
4
5  # Calcolo la massima similarità per ogni riga della matrice di similarità
6  for i in range(X_unlabeled.shape[0]):
7      max_sim[i] = np.max(similarity_matrix[i])
8
9  # Calcolo il valore medio delle massime similarità
10 mean_max_similarity = np.mean(max_sim)
11
12 print(f"Valore medio delle massime similarità: {mean_max_similarity}")

```

Il processo inizia con la creazione di un array `max_sim` inizializzato a zero, con una lunghezza pari al numero di righe di `X_unlabeled`, cioè il dataset non etichettato. Questo array sarà utilizzato per memorizzare la **massima similarità**

di ciascuna riga. La massima similarità rappresenta il valore più alto di similarità calcolato per ogni riga nella matrice di similarità, una tabella che misura la similarità tra coppie di oggetti nel dataset.

Successivamente, si itera su ciascuna riga della `similarity_matrix`. Per ogni riga, si calcola la similarità massima utilizzando la funzione `np.max` e si memorizza il risultato nell'array `max_sim`.

Infine, si calcola il **valore medio** delle massime similarità memorizzate in `max_sim` usando `np.mean` e si stampa il risultato. Questo valore medio viene utilizzato come **soglia massima** durante la creazione del dataset, aiutando a classificare correttamente le applet e a ridurre il rischio di falsi positivi.

3.1.4 Creazione del nuovo dataset

Al termine del processo descritto in 3.1.3, è stata determinata una soglia di similarità pari a `0.2721288492271225`. Questa soglia viene successivamente utilizzata per creare il dataset finale, etichettando tutte le applet in modo coerente con il livello di similarità rilevato.

```
1 # Vettorizzo le caratteristiche usando TF-IDF
2 vectorizer = TfidfVectorizer()
3 X_labeled = vectorizer.fit_transform(features_labeled.apply
4 (lambda x: ' '.join(x.astype(str)), axis=1))
5 X_unlabeled = vectorizer.transform(features_unlabeled.apply
6 (lambda x: ' '.join(x.astype(str)), axis=1))
```

Dopo aver importato i dataset etichettato e non etichettato in due DataFrame separati, procedo con la **vettorizzazione delle caratteristiche**. Questo processo trasforma variabili categoriali o testuali in rappresentazioni numeriche, consentendo l'analisi quantitativa delle applet.

```
1 # Calcola la similarità coseno tra tutte le coppie di applets
2 similarity_matrix = cosine_similarity(X_unlabeled, X_labeled)
3
4 # Imposta una soglia di similarità
5 similarity_threshold = 0.2721288492271225
```

Successivamente, viene creata la **matrice di similarità coseno**, che misura la similarità tra coppie di applet. La similarità coseno calcola il coseno dell'angolo tra i vettori rappresentanti le applet, con valori prossimi a 1 che indicano una forte somiglianza e valori vicini a 0 che indicano una bassa similarità.

```
1 # Predico le etichette per le applets nel dataset non etichettato
2 y_pred = []
3 for i in range(len(unlabeled_df)):
4     max_similarity = max(similarity_matrix[i])
5     if max_similarity >= similarity_threshold:
6         idx = list(similarity_matrix[i]).index(max_similarity)
7         predicted_label = labels_labeled[idx]
8     else:
9         predicted_label = 0 # Etichetta innocua
10
11     y_pred.append(predicted_label)
12
13 # Aggiungo le predizioni come colonna 'target'
14 unlabeled_df['target'] = y_pred
```

Una volta calcolata la matrice di similarità, per ogni riga della matrice viene ricavata la massima similarità con una delle applet nel dataset etichettato. A questo punto, si confronta tale valore con la soglia di similarità stabilita:

- Se il valore massimo di similarità supera la soglia, significa che l'applet non etichettata è molto simile a una già presente nel dataset. In tal caso, si assegna all'applet un'etichetta corrispondente a quella del dataset etichettato.
- Se la soglia non viene superata, l'applet viene considerata sufficientemente diversa dalle altre e le viene assegnata l'etichetta 0, che indica che l'applet è innocua.

Infine, le etichette predette per ciascuna applet non etichettata vengono raccolte in un vettore e aggiunte al dataset non etichettato, creando così il dataset finale arricchito con le nuove etichette. Questo processo permette di classificare automaticamente le applet, riducendo il rischio di falsi positivi e garantendo una maggiore accuratezza nella distinzione tra applet dannose e non dannose.

3.1.5 Analisi e ingegnerizzazione del nuovo dataset

Dopo aver creato il dataset come descritto nella sezione 3.1.4, ho deciso di raggruppare le applet in quattro gruppi distinti (uno per ciascun valore dell'etichetta) e generare un grafico che mostrasse la distribuzione delle etichette tra tutte le applet.

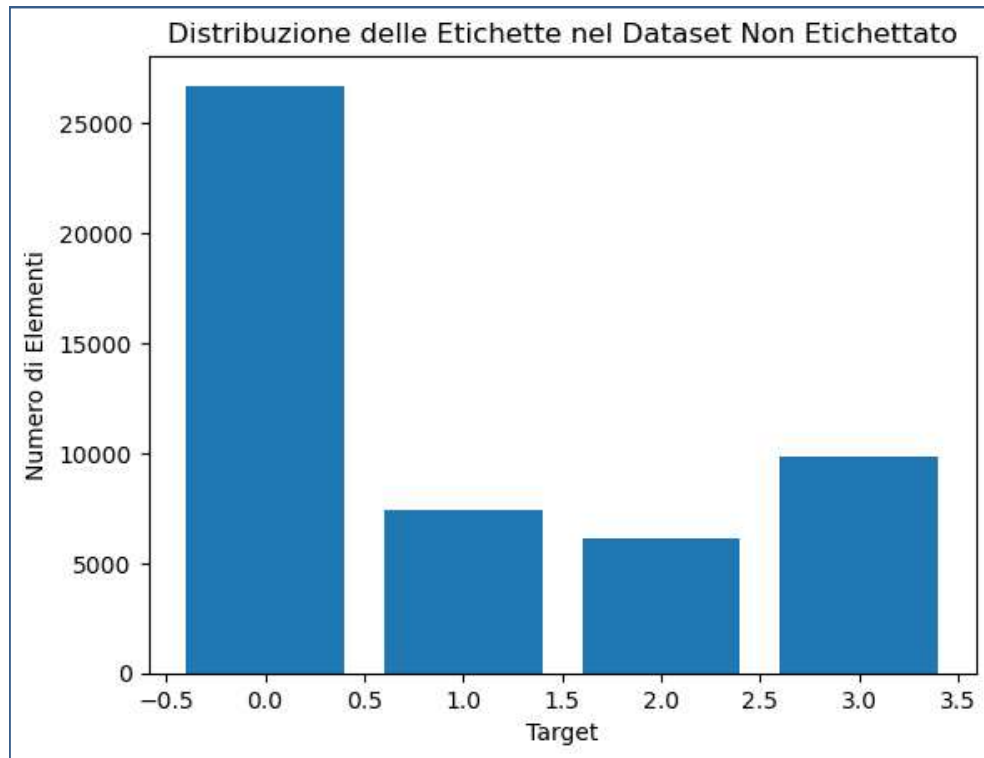


Figura 3.4: Distribuzione delle applet per etichetta

Come si può osservare nella Figura 3.4, il dataset generato mostra una netta predominanza di applet etichettate con il valore 0. Questo risultato è coerente con i dati riportati in [1] e [12], dove circa la metà delle applet risultano innocue, mentre l'altra metà presenta potenziali rischi.

Dopo aver analizzato la distribuzione delle etichette, ho proceduto con la fase di **ingegnerizzazione dei dati** per preparare il dataset all'addestramento dei modelli di machine learning. Il primo passo è stato eliminare le caratteristiche che non contribuivano significativamente alla **potenza predittiva**. Le caratteristiche mantenute sono:

- name
- installs_count

- services_len
- requires_android_app
- requires_ios_app
- requires_mobile_app
- triggers_category
- actions_category
- description
- service_triggers
- service_actions
- services
- by_service_owner
- target

Non ho seguito un criterio specifico per rimuovere le caratteristiche meno rilevanti; ho invece analizzato ogni feature singolarmente per decidere se fosse utile mantenerla.

Successivamente, ho effettuato una **pulizia del dataset**, eliminando tutte le applet con valori nulli o non disponibili (N/A).

Per evitare che il modello di machine learning sviluppasse una "preferenza" verso l'etichetta più rappresentata, ho applicato la tecnica di **undersampling** alla classe 0. Questo processo ha ridotto il numero di applet etichettate come innocue, permettendo di **equilibrare** meglio il dataset e prevenire squilibri nel modello predittivo.

```
1 # Ho diviso i dati per classe
2 df_class_0 = unlabeled_df_cleaned[unlabeled_df_cleaned['target'] == 0]
3 df_class_1 = unlabeled_df_cleaned[unlabeled_df_cleaned['target'] == 1]
4 df_class_2 = unlabeled_df_cleaned[unlabeled_df_cleaned['target'] == 2]
5 df_class_3 = unlabeled_df_cleaned[unlabeled_df_cleaned['target'] == 3]
6
```

```

7 # Ho calcolato la dimensione media delle classi 1, 2 e 3
8 n_avg = int((len(df_class_1) + len(df_class_2) + len(df_class_3)) / 3)

```

Per ottenere un dataset bilanciato, ho suddiviso il dataset in 4 classi, una per ciascun valore dell'etichetta. Successivamente, ho calcolato la dimensione media delle classi con etichette diverse da 0. Questo passaggio è fondamentale per completare il processo di **sottocampionamento** (*undersampling*), poiché consente di indicare alla funzione `resample` fino a quale punto ridurre il numero di applet etichettate come innocue, garantendo così un dataset più bilanciato per l'addestramento del modello.

```

1 # Sotto-campionamento della classe 0
2 df_class_0_under = resample(df_class_0,
3                             replace=False,
4                             n_samples=n_avg,
5                             # per abbinare la dimensione media delle
6                             # altre classi
7                             random_state=42)
8                             # per la riproducibilità

```

Infine, ho ripetuto l'analisi descritta all'inizio del paragrafo, ottenendo un bilanciamento corretto tra le classi, come mostrato nella Figura 3.5.

3.2 Processo di Addestramento del Modello

Dopo aver completato la fase di ingegnerizzazione dei dati, come descritto in 3.1.5, ho dovuto selezionare l'algoritmo più adatto al mio dataset, cercando un equilibrio tra efficienza e ottime metriche di valutazione.

Per farlo, ho adottato un approccio empirico, testando una serie di algoritmi sia di classificazione che di regressione. Per organizzare il processo, ho creato una classe dedicata che svolgeva le seguenti funzioni principali:

1. Preparare i dati per l'addestramento
2. Ottimizzare i parametri del modello attraverso la ricerca di iperparametri
3. Addestrare il modello selezionato
4. Valutare le performance del modello

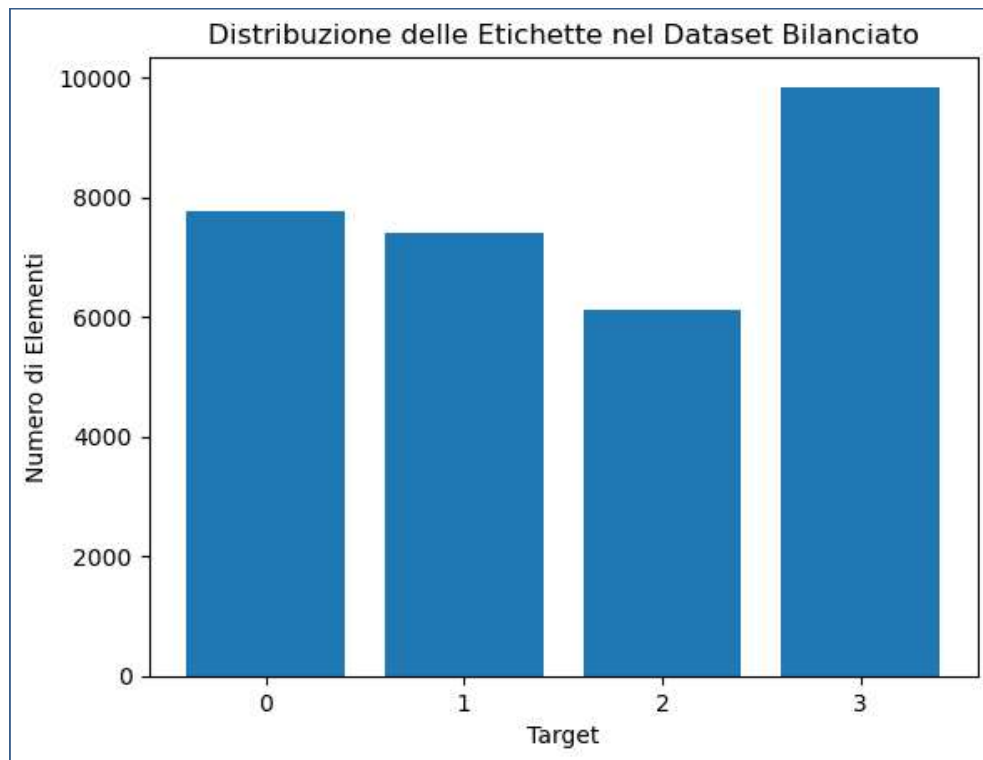


Figura 3.5: Distribuzione delle applet per etichette dopo il bilanciamento

La preparazione dei dati includeva una serie di istruzioni aggiuntive per garantire che fossero adeguatamente strutturati e pronti per l'addestramento e la successiva valutazione dei modelli.

```

1 X = self.data.drop(columns=[self.target_column])
2 y = self.data[self.target_column]
3
4 # Riduco il numero di istanze (campionamento del 10%)
5 X_sampled = X.sample(frac=self.sample_fraction, random_state=42)
6
7 # Concateno tutte le colonne in un unico testo per ogni riga
8 X_text = X_sampled.apply(lambda row: ''
9     .join(row.values.astype(str)), axis=1).values
10
11 # Creazione del vettore TF-IDF per le feature testuali
12 X_tfidf = self.vectorizer.fit_transform(X_text)
13
14 # Utilizzo SelectKBest con il test del chi-quadrato per
15 # selezionare le migliori caratteristiche
16 selector = SelectKBest(score_func=chi2, k=10000)
17 X_selected = selector.fit_transform(X_tfidf, y_sampled)
18

```



```
19 # Applico la standardizzazione ai dati numerici
20 scaler = StandardScaler()
21 X_selected_scaled = scaler.fit_transform(X_selected.toarray())
22
23 # Split del dataset in addestramento e test
24 X_train, X_test, y_train, y_test =
25 train_test_split(X_selected_scaled, y_sampled,
26 test_size=self.test_size, random_state=42)
27
28 return X_train, X_test, y_train, y_test
```

Per prima cosa, ho suddiviso il dataset verticalmente, separando la variabile target dalle feature indipendenti. A causa delle limitate risorse computazionali a mia disposizione, ho deciso di ridurre il numero di istanze al 10

Successivamente, per trasformare le feature testuali in valori numerici, ho concatenato tutte le feature in un'unica riga testuale e poi effettuato la vettorizzazione. La vettorizzazione converte il testo o le variabili categoriali in rappresentazioni numeriche, creando una feature per ogni parola o categoria distinta. Questo processo aumenta il numero di feature, poiché ogni elemento unico diventa una variabile separata.

Per evitare sovraccarichi computazionali durante l'addestramento, ho selezionato le migliori 10.000 feature, applicando successivamente lo scaler `StandardScaler`. Questo metodo normalizza i dati, trasformandoli in modo che abbiano media 0 e deviazione standard 1. Ho scelto `StandardScaler` dopo aver confrontato empiricamente altri scaler, constatando che, pur non influenzando significativamente le prestazioni del modello, offriva un leggero miglioramento in termini di efficienza.

Passando ai metodi di addestramento, come ho accennato prima ho voluto provare ogni combinazione di parametri del modello dati in input per estrarre la combinazione migliore. Il tutto viene mostrato dal seguente codice.

```
1 # Usa RandomizedSearchCV per trovare i migliori parametri
2 random_search = RandomizedSearchCV(estimator=model,
3 param_distributions=param_grid, n_iter=100, cv=5, verbose=2,
4 random_state=42, n_jobs=-1)
5
6 # Esegui la ricerca sui dati di addestramento
7 random_search.fit(X_train, y_train)
```

```
8
9 # Ottieni i migliori parametri trovati
10 best_params = random_search.best_params_
11 print("Migliori parametri trovati:", best_params)
```

Per la selezione degli iperparametri, ho impiegato **RandomizedSearchCV**, uno strumento particolarmente efficiente quando si ha a che fare con spazi di ricerca molto ampi. A differenza di **GridSearchCV**, che esplora tutte le possibili combinazioni, **RandomizedSearchCV** seleziona casualmente una serie di combinazioni, riducendo così il tempo di calcolo pur mantenendo elevate probabilità di individuare i parametri ottimali.

Una volta identificati i migliori iperparametri e salvati in `best_params`, ho addestrato il modello utilizzando tali parametri sul **training set**, una porzione di dati utilizzata per consentire al modello di apprendere le relazioni tra le caratteristiche e le etichette.

Per valutare le prestazioni dei diversi modelli, ho creato un metodo che calcola e stampa i seguenti parametri di valutazione:

- **Accuracy:** È la percentuale di previsioni corrette sul totale delle previsioni effettuate. Misura la capacità del modello di classificare correttamente sia le classi positive che negative.
- **Precision (precisione):** Indica la proporzione di previsioni corrette tra quelle che sono state classificate come positive. Una precisione alta significa che pochi falsi positivi sono stati classificati come veri.
- **Recall (richiamo):** Indica quante delle osservazioni effettivamente positive sono state correttamente identificate dal modello. Un recall alto significa che pochi falsi negativi sono stati commessi.
- **F1 Score:** È la media armonica tra precisione e recall. Viene usato quando si vuole bilanciare precisione e richiamo, specialmente in situazioni con distribuzioni sbilanciate delle classi.

- **Confusion Matrix (matrice di confusione):** È una tabella che riassume le prestazioni del modello mostrandone i veri positivi (TP), veri negativi (TN), falsi positivi (FP) e falsi negativi (FN).

3.2.1 Applicazione del metodo empirico

Una volta creata la classe descritta in 3.2, l'applicazione del metodo empirico è risultata notevolmente semplificata. Per metodo empirico intendo la selezione e valutazione di diversi algoritmi di machine learning (ML), con l'obiettivo di identificare quelli che offrono il miglior compromesso tra **efficienza** e **prestazioni**. Gli algoritmi selezionati per la fase successiva di valutazione sono stati sottoposti a una valutazione completa utilizzando l'intero dataset (per ulteriori dettagli, vedere il paragrafo 3.2.2).

Il processo è piuttosto diretto: per ciascun algoritmo, ho esplorato diverse combinazioni di iperparametri e successivamente ho utilizzato le funzioni descritte in 3.2 per completare l'addestramento e la valutazione del modello.

Sebbene gli algoritmi di **classificazione** siano particolarmente adatti per questo problema, ho deciso di includere anche algoritmi di **regressione** per esaminare come si comportano con un dataset etichettato con valori discreti.

In particolare, per gli algoritmi di classificazione ho considerato:

- **Support Vector Machine (SVM):** L'SVM è un algoritmo di classificazione che cerca di trovare un iperpiano ottimale che separi i dati in classi distinte. È particolarmente utile con dati testuali vettorizzati perché gestisce bene spazi ad alta dimensionalità e garantisce buone prestazioni su dataset con molte feature sparse, tipici della vettorizzazione del testo.
- **Random Forest:** Random Forest è un ensemble di alberi decisionali che crea un modello robusto combinando molteplici classificatori (alberi) e basandosi su un processo di voto per ottenere il risultato finale. È scelto per la sua capacità di gestire feature ad alta dimensionalità e per la sua robustezza contro l'overfitting, rendendolo adatto per dati testuali complessi.

- **XGBoost:** XGBoost è un algoritmo di boosting basato su alberi che migliora le previsioni combinando modelli deboli in una sequenza iterativa. È noto per la sua efficienza e precisione, ed è particolarmente efficace su dati testuali vettorizzati grazie alla sua capacità di catturare interazioni complesse tra le feature e di gestire grandi volumi di dati.
- **K-Nearest Neighbors (KNN):** KNN è un algoritmo di classificazione basato sulla distanza che assegna la classe di un'osservazione in base alle classi dei suoi vicini più prossimi. Sebbene sia semplice e intuitivo, può essere computazionalmente costoso con dati testuali ad alta dimensionalità; tuttavia, è utile per valutare la similarità tra istanze in uno spazio vettorizzato.
- **Logistic Regression:** La regressione logistica è un algoritmo di classificazione che utilizza una funzione logistica per modellare la probabilità che un'osservazione appartenga a una determinata classe. È particolarmente utile per i dati testuali vettorizzati perché gestisce bene le relazioni tra le variabili indipendenti e la probabilità di appartenenza a una classe, risultando efficace anche con grandi spazi di feature.

Per gli algoritmi di regressione invece ho considerato:

- **3.3.2 Linear Regression:** La regressione lineare è un metodo di regressione che modella la relazione tra una variabile dipendente e una o più variabili indipendenti tramite una funzione lineare. È scelta per la sua semplicità e interpretabilità, ed è adatta per predire valori continui, anche se le feature vettorizzate possono aumentare la complessità del modello.
- **3.3.3 Ridge Regression:** La Ridge Regression è una variante della regressione lineare che include un termine di penalizzazione L2 per ridurre la complessità del modello e prevenire l'overfitting. È particolarmente utile quando si lavora con dati ad alta dimensionalità, come le feature vettorizzate dei testi, poiché stabilizza le stime dei coefficienti e migliora la generalizzazione del modello.

3.2.2 Algoritmi scelti e fasi conclusive

Da quanto osservato, gli algoritmi di regressione non hanno raggiunto le prestazioni degli algoritmi di classificazione. Tra gli algoritmi di classificazione considerati, ho scelto **Logistic Regression** e **Support Vector Machine (SVM)**. Entrambi hanno mostrato risultati equivalenti con una precisione, recall, accuracy e F1 Score del **86%**. Tuttavia, **Logistic Regression** ha dimostrato una **maggiore velocità di predizione**.

Per ottenere una valutazione più rappresentativa, ho eseguito una **seconda valutazione** utilizzando l'intero dataset e i parametri ottimizzati salvati in `best_params`. Questo approccio ha garantito un addestramento più **efficiente** e risultati più affidabili.

Inaspettatamente, **SVM** si è rivelato **più performante** rispetto a **Logistic Regression**, con un miglioramento del **3%** in tutti i parametri di prestazione. Nonostante **Logistic Regression** fosse più veloce nella valutazione, ho scelto **SVM** per la sua superiorità nelle prestazioni complessive.

Successivamente, ho esportato l'algoritmo **SVM** come file **PMML** [4] per integrarlo in **IFTTT Security Extension** (3.3). Utilizzando **Spring Boot** e le librerie `pmml-evaluator` e `pmml-evaluator-extension`, ho potuto non solo esportare il modello, ma anche effettuare le **predizioni**.

3.3 Ingegneria di IFTTT Security Extension

IFTTT Security Extension è il nome dato alla web app che integra il modello di machine learning discusso in 3.2. L'applicazione ha attraversato una fase di ingegneria del software che l'ha portata a diventare una piattaforma facilmente accessibile per la maggior parte degli utenti IFTTT.

Nei seguenti paragrafi verranno trattati i principali aspetti del processo di ingegneria del software, con esclusione della valutazione dell'usabilità, posticipata a revisioni future.

3.3.1 Analisi

L'analisi del software ha permesso di delineare chiaramente l'obiettivo del sistema e, in maniera astratta, le funzionalità necessarie per garantire un'esperienza completa.

L'obiettivo principale era creare una piattaforma usabile che consentisse agli utenti di eseguire una predizione con poche interazioni (RQ3):. Date le elevate percentuali di installazioni di applet dannose (citato in 3.1.2), si ipotizza che gli utenti IFTTT spesso non si preoccupano dei rischi legati all'installazione di applet dannose. Pertanto, l'interesse verso un'applicazione come **IFTTT Security Extension** potrebbe diminuire rapidamente se non opportunamente incentivato.

Requisiti funzionali

A causa delle limitate risorse computazionali, è stato deciso di predire **solo applet** già assemblate e presenti sul sito di IFTTT³. Tuttavia, si è voluto comunque predisporre una base per aggiornamenti futuri che permettessero anche la **composizione** delle applet, consentendo agli utenti di scegliere *trigger* e *action*.

Nonostante questa limitazione, l'intenzione era di consentire agli utenti di effettuare predizioni in più modi:

- Tramite una barra di ricerca per selezionare l'applet desiderata.
- Permettere una ricerca basata su uno dei due servizi/canali coinvolti nell'applet, nel caso non fossero noti tutti i dettagli.

Inoltre, è stato pensato di includere una pagina che mostri una tabella con tutte le applet predette e una sezione che informi l'utente sui potenziali rischi derivanti dall'attivazione dell'applet, spiegando anche la natura del problema (RQ4):.

Questo ha portato alla definizione dei seguenti requisiti funzionali, riassunti nella tabella 3.1.

Requisiti non funzionali

Come accennato in ??, l'obiettivo era creare una piattaforma altamente **usabile** per mantenere vivo l'interesse degli utenti di IFTTT. Un aspetto strettamente correlato

³Link del sito: <https://ifttt.com/explore/services>

all'usabilità è l'**efficienza**. La fluidità nell'interazione, come quella che si riscontra in un IDE⁴, contribuisce a rendere l'esperienza d'uso più piacevole.

Tuttavia, la quantità di dati da recuperare e gestire era **considerevole**, mettendo sotto pressione le risorse computazionali a mia disposizione. Per evitare che la risoluzione di bug o piccoli aggiustamenti diventasse inefficiente a causa di processi troppo lunghi, ho scelto di suddividere il progetto in diversi **moduli indipendenti**. Questa scelta ha migliorato notevolmente la **manutenibilità** del software, creando però un **trade-off** tra i requisiti di usabilità ed efficienza.

Storyboard

Lo storyboard è uno strumento utilizzato per pianificare e visualizzare il design e il flusso di un sito o di un'applicazione web. Rappresenta una serie di schizzi o diagrammi che illustrano il percorso che gli utenti seguono attraverso le diverse pagine e interazioni [?].

Nel contesto della realizzazione di IFTTT Security Extension, lo storyboard è stato fondamentale per analizzare i migliori percorsi di navigazione e per gestire correttamente le pagine web, mantenendo un'elevata usabilità. L'obiettivo principale era creare **poche pagine ricche di contenuti**, ma bilanciare l'abbondanza delle informazioni con la leggibilità delle pagine stesse era una sfida chiave.

Per evitare il sovraccarico visivo e garantire una buona user experience, ho scelto di adottare una soluzione che prevedeva l'uso di diversi **pop-up**, che apparissero in primo piano, oscurando il resto della pagina e fungendo da sfondo. Questo approccio ha consentito di mantenere le informazioni essenziali a portata di mano senza compromettere la semplicità di navigazione.

Lo storyboard progettato è illustrato nella figura 3.6.

3.3.2 Progettazione di alto livello

In questa sezione verrà descritta la **struttura** del progetto e le scelte **architettoniche** adottate.

⁴Un IDE (Integrated Development Environment) è un software che fornisce strumenti integrati per lo sviluppo, la scrittura, il debug e la gestione di codice in diversi linguaggi di programmazione.

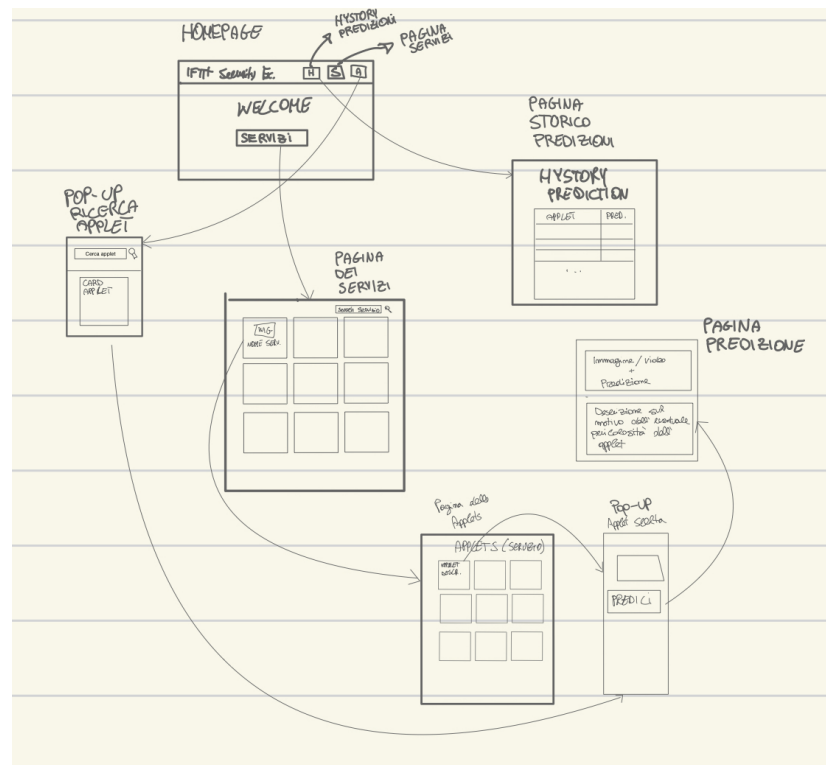


Figura 3.6: Storyboard IFTTT Security Extension

Come discusso nel paragrafo 3.3.1, uno dei requisiti non funzionali principali era la **manutenibilità**. Per soddisfare questo requisito, ho deciso di suddividere il progetto in **quattro moduli indipendenti**, di cui due utilizzati per popolare il database con le informazioni provenienti da IFTTT e gli altri due impiegati per lo sviluppo dell'applicazione vera e propria. I moduli sono:

- **LinkLoader**: Utilizzato per recuperare e talvolta **costruire** i link delle pagine web da cui estrarre informazioni.
- **DatabaseLoader**: Responsabile del **Web Scraping**, ovvero del recupero delle informazioni dalle pagine dei link ottenuti dal modulo LinkLoader (RQ5):.
- **Backend**: Implementa la **logica applicativa** della web app, gestendo l'interazione con il database e il modello di Machine Learning.
- **Frontend**: Utilizzato per sviluppare l'interfaccia e gestire le funzionalità legate al **rendering dinamico** delle pagine.

Nonostante i quattro moduli siano **indipendenti**, essi **cooperano** tra loro, assicurando il corretto funzionamento dell'intero sistema. In linea con questa struttura, ho adottato l'architettura **client/server**, dove il client (modulo `Frontend`) esegue **chiamate API** al server (modulo `Backend`).

Per quanto riguarda la struttura interna dei vari sottosistemi, i moduli `LinkLoader` e `DatabaseLoader` non richiedevano una ulteriore suddivisione poiché i loro compiti erano chiaramente definiti. Il `Backend` e il `Frontend`, se considerati come un unico **macrosistema**, evidenziavano i seguenti sottosistemi:

- **Gestione dei servizi:** Si occupa di tutte le funzionalità legate ai servizi, come il recupero, la visualizzazione e il filtraggio.
- **Gestione delle applet:** Gestisce le operazioni relative alle applet, come la ricerca, il filtraggio e la visualizzazione.
- **Gestione delle predizioni:** Integra il modello di Machine Learning per effettuare predizioni sulle applet, mostrando i relativi rischi agli utenti.

Dal punto di vista della **struttura "orizzontale"**, per ogni modulo (ad eccezione del `Frontend`), ho applicato il design pattern **Service Layer** (per ulteriori dettagli si veda 3.3.3). Questo pattern organizza il codice in tre livelli distinti:

- **Controller:** Gestisce le richieste dell'utente e fornisce le risposte appropriate.
- **Service:** Implementa la logica applicativa.
- **Entity:** Gestisce gli oggetti persistenti del sistema e fornisce le operazioni legate a tali oggetti.
- **Persistence:** Include tutti i dati persistenti del sistema.

L'utilizzo del pattern `Service Layer` consente una maggiore organizzazione del codice e facilita ulteriormente la **manutenibilità** del sistema.

Persistenza dei dati

Un aspetto fondamentale della progettazione riguarda la **persistenza dei dati**. Come accennato in precedenza, mi servivano informazioni relative a **applet** e **servizi**. Per i servizi, erano sufficienti informazioni di base, mentre per le applet erano necessarie informazioni più dettagliate.

Un'applet è composta da un **Trigger** e una **Action**, pertanto è stato necessario strutturare correttamente le informazioni relative a queste *tre* entità. Inoltre, il modello di Machine Learning richiedeva informazioni sul **creatore** dell'applet, che ho trattato come un'entità a parte per evitare **ridondanza** di dati nel database.

Sulla base di questa analisi, ho progettato il **modello E/R** ⁵ rappresentato nella figura 3.7.

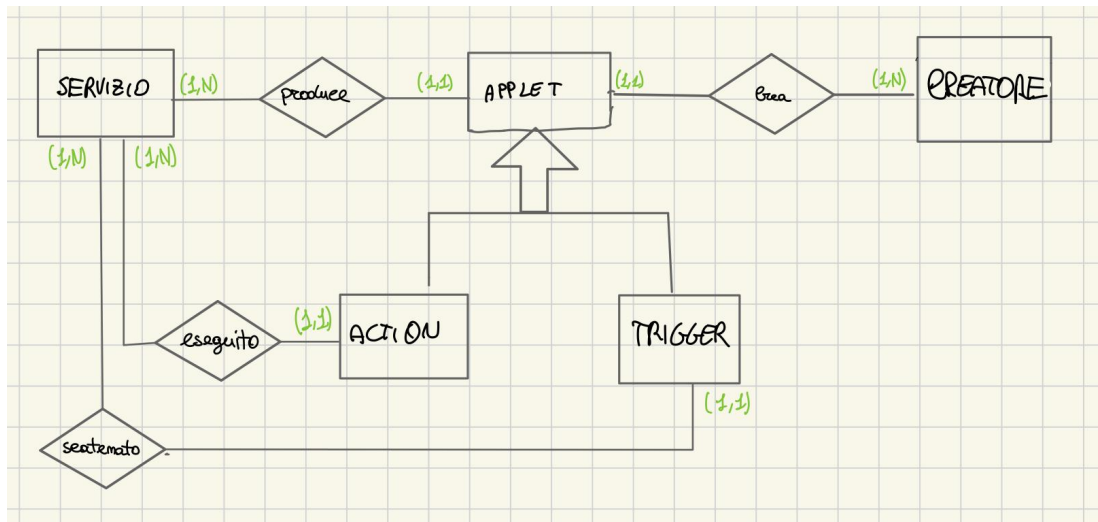


Figura 3.7: Modello E/R di IFTTT Security Extension

⁵Un modello E/R (Entità-Relazioni) è una rappresentazione grafica che descrive le entità di un sistema, i loro attributi e le relazioni tra di esse.

3.3.3 Progettazione di basso livello

In questa sezione approfondiremo le scelte progettuali legate all'implementazione, focalizzandoci su soluzioni che soddisfano i requisiti funzionali e non funzionali.

Component Off-The-Shelf (COTS)

I **COTS** (Commercial Off-The-Shelf) sono prodotti software o hardware già pronti all'uso, sviluppati per il mercato di massa e facilmente integrabili nei progetti senza necessitare di personalizzazioni significative. Questi componenti vengono scelti per ridurre i costi e i tempi di sviluppo.

Per lo sviluppo di questa applicazione sono stati adottati i seguenti COTS:

- **Backend:** È stato utilizzato **Spring Boot** per lo sviluppo del lato server, con l'interazione che avviene tramite il linguaggio di programmazione **Java**. Spring Boot è stato scelto per la sua **efficienza** e **comodità**, in quanto offre un **container**⁶ che gestisce automaticamente operazioni critiche, come la gestione della sincronizzazione con il database.
- **Frontend:** La libreria **ReactJS** è stata scelta per la progettazione del lato client. ReactJS consente di creare interfacce utente altamente reattive, migliorando l'**usabilità** e l'**efficienza** dell'applicazione grazie alla velocità di navigazione e al rendering dinamico delle pagine. Per lo stile grafico, è stata utilizzata la libreria **Chakra UI**.
- **Database:** I dati sono stati memorizzati all'interno di un **DBMS MySQL**, utilizzando **JPA** (*Java Persistence API*), con il framework Spring Boot che adotta la tecnologia **Hibernate** per la gestione della persistenza. La connessione tra l'applicazione e il database è garantita dall'uso del driver **JDBC**.

Design Patterns

Nella progettazione ad alto livello, ho descritto il design pattern **Service Layer**, che aiuta a organizzare i servizi del sistema in livelli distinti e indipendenti. Ogni

⁶Un container è un'unità software leggera che racchiude un'applicazione e tutte le sue dipendenze, garantendone l'esecuzione isolata su qualsiasi ambiente.

livello fornisce un'interfaccia chiara al livello superiore, mentre il suo output diventa l'input per il livello inferiore [?]. 3.8

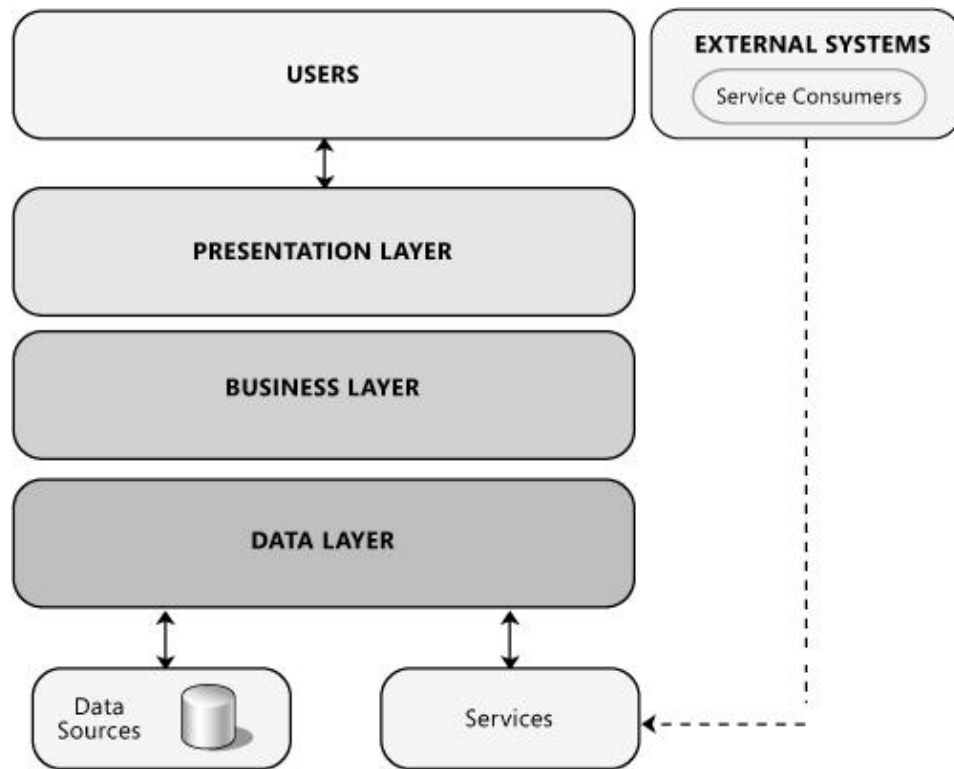


Figura 3.8: Modello E/R di IFTTT Security Extension

Un altro pattern che ho utilizzato è il **Repository**. Questo pattern, come descritto nella documentazione di Spring e nelle best practices per applicazioni Enterprise, separa la logica di accesso ai dati dalle entità di business. La comunicazione tra la logica di accesso ai dati e la logica di business avviene tramite **interfacce**, garantendo una gestione sicura e debolmente accoppiata con il DBMS. 3.9

Organizzazione delle Operazioni

Per migliorare l'**efficienza** dell'applicazione, ho implementato le funzionalità in modo da ottimizzare le interazioni tra il frontend e il backend. Essendo l'architettura del sistema basata su client/server, il client deve effettuare **chiamate API** al server per ottenere le informazioni da visualizzare. Questo processo richiede tempo e può influire sull'efficienza complessiva dell'applicazione. Pertanto, è stato essenziale

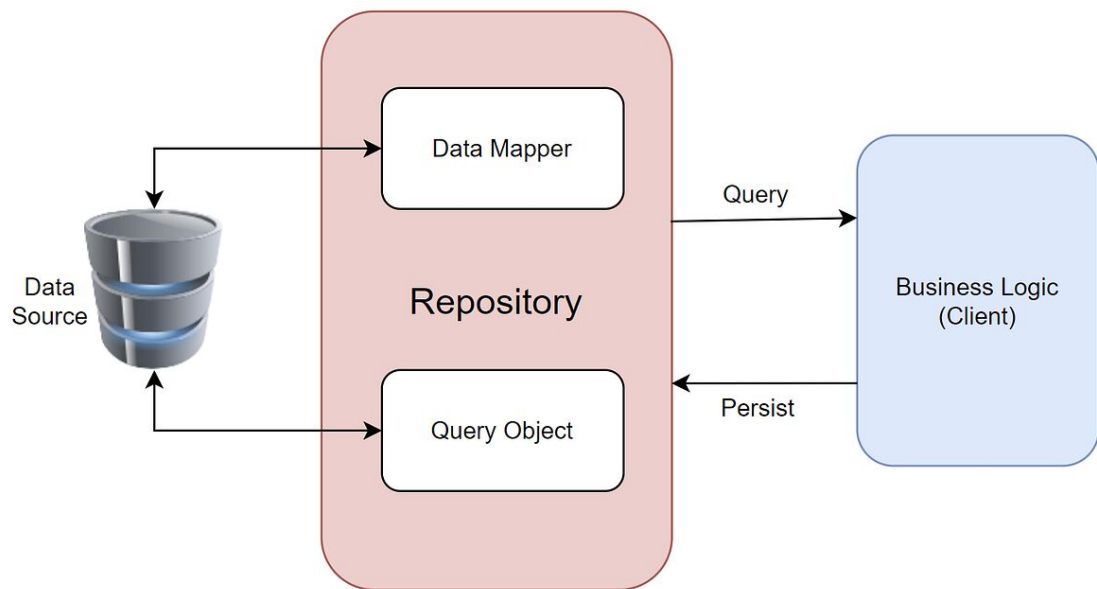


Figura 3.9: Modello E/R di IFTTT Security Extension

organizzare le operazioni per ridurre il numero di chiamate API e migliorare le performance.

Per minimizzare le chiamate API, ho progettato il sistema per eseguire tali chiamate solo quando necessario, come ad esempio quando i dati non sono già memorizzati in sessione. Ad esempio, per il filtraggio dei servizi, il client prima deve ottenere l'elenco completo dei servizi tramite l'API `ottieni servizi`. Una volta ottenuti, i servizi vengono memorizzati in sessione o in una memoria con uno scope più ampio di una singola richiesta HTTP. Questo approccio riduce la necessità di effettuare chiamate API ripetute per applicare filtri sui servizi già ottenuti, migliorando così l'efficienza.

Tuttavia, è importante considerare le risorse hardware disponibili, come la **memoria**, per evitare un eccessivo riempimento dei buffer.

Ho organizzato le operazioni tra frontend e backend, associando le seguenti operazioni alle API del backend:

- **Ricerca applet:** Questa API accetta il **nome dell'applet** come input e restituisce

una **lista di applet** corrispondenti. Questa operazione è utile quando il client non ha accesso immediato alle applet e deve eseguire una ricerca diretta.

- **Stampa servizi:** Questa API effettua una richiesta GET⁷ per ottenere una **lista di servizi**.
- **Ricerca applet per servizio:** Questa API richiede il **codice identificativo** di un servizio e restituisce una **lista di applet** associate a quel servizio. È necessaria quando il client deve recuperare informazioni sulle applet legate a un servizio specifico.
- **Predizione sicurezza applet:** Questa API fornisce una predizione del livello di sicurezza di un'applet, restituendo un **numero da 0 a 3** che indica il grado di sicurezza, come descritto in 3.1.2.

3.4 Modulo LinkLoader

Il modulo LinkLoader è stato progettato per raccogliere in modo efficiente i link delle pagine web da sottoporre alla fase di web scraping. La necessità di creare un modulo dedicato è nata dalla volontà di ottimizzare l'uso delle risorse computazionali disponibili, data la grande quantità di link da processare. Il volume di dati raccolti è stato infatti molto elevato, rendendo l'intero processo particolarmente lungo, con una durata complessiva di **3-4 ore**.

Il funzionamento del LinkLoader inizia con l'accesso alla pagina `https://ifttt.com/exp` dove sono elencati tutti i **servizi partner** che hanno consentito l'attivazione di applet che li coinvolgono. A partire da questa pagina, il modulo ha estratto il contenuto del DOM⁸, esaminando l'HTML per individuare gli URL dei servizi. Per farlo, ha riconosciuto e sfruttato pattern comuni nella struttura dinamica delle pagine di IFTTT.

⁷La richiesta GET è un metodo HTTP utilizzato per richiedere dati da un server senza modificare le risorse, inviando i parametri nella URL.

⁸Il DOM (Document Object Model) è una rappresentazione strutturata di un documento HTML o XML che consente ai linguaggi di programmazione, come JavaScript, di interagire e modificare dinamicamente il contenuto, la struttura e lo stile di una pagina web.

I link estratti sono stati quindi salvati in un file JSON, successivamente utilizzato per individuare e costruire i collegamenti alle applet associate ai singoli servizi.

Per rendere il processo più completo e facilmente estendibile a future versioni dell'applicazione, il LinkLoader non si è limitato a raccogliere i link delle applet. Al termine dell'estrazione, ha infatti proseguito con la raccolta dei link relativi ai **trigger** e alle **actions**, elementi strettamente legati ai servizi di cui si stava effettuando la navigazione.

In sintesi, questo modulo ha automatizzato l'intero processo di navigazione sul sito IFTTT, partendo dai singoli servizi per arrivare all'estrazione sistematica dei link di applet, trigger e actions. Il risultato è un file JSON (`links.json`) contenente i collegamenti a questi elementi, pronti per essere utilizzati nelle fasi successive del progetto.

3.5 Modulo DatabaseLoader

Questo modulo è progettato per popolare il database utilizzando le informazioni raccolte dagli URL salvati nel file `links.json`, come descritto nel paragrafo 3.4.

Il processo di popolamento del database segue una procedura simile a quella del modulo precedente: consiste nel navigare su pagine web e estrarre porzioni specifiche di HTML. Per questo, è stata necessaria una fase preliminare di **studio** delle pagine web, al fine di identificare i marcatori e le classi pertinenti per l'estrazione dei dati.

Un problema significativo emerso durante questo processo è stata la **lentezza di esecuzione**. Con oltre 10.000 applet e servizi da estrarre, il rischio di dover attendere ore per ogni errore nella fase di web scraping era elevato. Per affrontare questo problema, è stata adottata una strategia di **salvataggio delle porzioni HTML** in file JSON. Questo approccio ha permesso di separare la fase di web scraping dalla fase di popolamento del database, facilitando la verifica delle informazioni estratte e consentendo di identificare rapidamente eventuali errori senza impattare il database.

Sono stati creati tre file JSON per gestire le informazioni estratte:

- `applets.json`: Contiene le informazioni preliminari delle applet, come il *nome*, la *descrizione*, e il *nome del creatore*.

- `details.json`: Salva i dettagli delle applet, comprendenti informazioni su trigger e azioni.
- `services.json`: Contiene le informazioni sui servizi.

Durante la realizzazione del modulo, è stato deciso di suddividere il web scraping in **tre macro-operazioni**, ognuna delle quali si occupa di una tipologia specifica di informazioni. Ogni operazione inizia con il web scraping e termina con il processamento dei dati e l’inserimento degli elementi nel database. Per gestire queste operazioni in modo efficiente, è stata creata una classe Main su Spring Boot che accetta un parametro per avviare una delle tre tipologie di esecuzioni. Queste esecuzioni sono state suddivise in metodi controller, che richiamano i metodi dei layer sottostanti.

Le prime due operazioni hanno consentito la creazione di istanze nel database. Tuttavia, per l’operazione riguardante i dettagli, è stato necessario eseguire operazioni di `update`, poiché alcune informazioni non riguardavano solo trigger e azioni, ma anche dettagli aggiuntivi sulle applet.

3.6 Integrazione del modello

Come specificato in precedenza, il classificatore è stato esportato come un file **PMML** per consentire al modulo **backend** di integrarlo all’interno dell’applicativo.

Prima di procedere con l’integrazione, è stato necessario gestire l’aggiunta delle feature derivanti dal processo di **vettorizzazione**. Come indicato nei paragrafi precedenti, la vettorizzazione è stata applicata per il trattamento delle feature testuali, portando alla creazione di **migliaia di feature**, difficilmente gestibili all’interno dell’applicativo.

3.6.1 Modalità di estrazione del modello

Prima dell’esportazione, ho creato un file per **contare** il numero di feature generate per ciascuna variabile testuale. Per ragioni di efficienza, ho limitato l’aggiunta di feature agli attributi `nome` e `descrizione`, fino a un massimo di 5000 ciascuno.

Questo processo è servito anche per mantenere sotto controllo le dimensioni del dataset. La vettorizzazione ha generato complessivamente 11887 feature.

Successivamente, è stato necessario assegnare un **nome** a ciascuna feature, in modo da consentire al modulo backend di gestire correttamente gli input da fornire al modello per effettuare le predizioni.

```
1
2 # Carico i componenti della pipeline dal file
3 'feature_counts.json'
4 with open('feature_counts.json', 'r') as f:
5     feature_counts = json.load(f)
6
7 # Creazione dei nomi dei campi attivi basati su feature_counts
8 active_fields = []
9 for col, num_features in feature_counts.items():
10     for i in range(num_features):
11         active_fields.append(f"{col}_{i+1}")
12
13 # Aggiungo ogni campo da numerical_features e
14 # categorical_features a active_fields
15 for feature in numerical_features:
16     active_fields.append(feature)
17
18 for feature in categorical_features:
19     active_fields.append(feature)
```

Inizialmente, il file `feature_counts.json` viene letto per ottenere un oggetto che associa le variabili testuali al numero di feature generate. Questo file è stato creato durante la fase di addestramento del classificatore **SVM**, al termine della vettorizzazione.

Successivamente, viene creato un vettore vuoto, che verrà riempito con 11887 elementi, uno per ciascuna feature. All'interno del ciclo, vengono generate stringhe formate dal nome della feature estratta da `feature_counts`, seguite da un indice incrementale. Ad esempio, se il campo `nome` ha generato 5000 feature, nel vettore saranno presenti 5000 stringhe denominate: `nome1`, `nome2`, ..., `nome5000`.

Dopo aver creato i nomi dei campi basati su `feature_counts`, il codice aggiunge anche i nomi dei campi dalle liste `numerical_features` e `categorical_features`. Queste liste contengono rispettivamente le feature numeriche (es. `installs_count`)

e categoriche (es. `target`) già identificate, che vengono aggiunte direttamente alla lista `active_fields`. Questo permette di ottenere un elenco completo di **tutte le feature** del dataset, che saranno successivamente utilizzate come **campi attivi** per eseguire la predizione.

```

1  # Definisco una pipeline con solo il classificatore
2  pipeline = PMMLPipeline([
3      ("classifier", svm_model)
4  ])
5
6  # Definizione esplicita dei campi target e attivi
7  pipeline.target_fields = ['target']
8  pipeline.active_fields = active_fields
9
10 # Esporto il modello in formato PMML
11 try:
12     sklearn2pmml(pipeline, "ClassificatoreSicurezzaApplets.pmml",
13                 with_repr=True)
14     print("Modello PMML esportato con successo.")
15 except Exception as e:
16     print("Errore durante l'esportazione del modello PMML:", e)

```

Infine, viene eseguita l'esportazione del modello in formato PMML per poterlo integrare con il backend.

3.6.2 Gestione delle predizioni

Durante la progettazione ad alto livello, è stato previsto un sottosistema dedicato alla gestione delle operazioni legate al classificatore SVM. Questo sottosistema è composto da varie classi, tra cui la classe `PredictionController`, che ha il compito di gestire le richieste di predizione da parte dell'utente. Il `PredictionController` riceve come input l'**identificativo** dell'applet e restituisce come output la predizione sotto forma di un **valore numerico**.

Per eseguire la predizione, il `PredictionController` invoca la classe `PredictionService`. Quest'ultima non si occupa solo di interagire con il modello PMML, ma prepara anche l'input necessario al modello, componendo una **lista di item**, in modo simile a quanto descritto nel paragrafo precedente.

Nel dettaglio, la `PredictionService` crea una *Map* che contiene tutti gli input richiesti dal modello. L'uso della struttura *Map* è funzionale per associare le coppie (nome variabile, numero di feature), informazioni che vengono recuperate sia dal database (durante la ricerca tramite identificativo), sia dal file `feature_counts.json` per ottenere il numero corretto di feature.

Successivamente, la classe gestisce la vettorizzazione delle feature testuali, definendo correttamente i **nomi** e il **numero** delle feature, come è stato fatto durante l'esportazione del modello. Una volta che l'input è stato preparato, la predizione viene eseguita interagendo direttamente con il modello PMML tramite apposite funzioni di libreria.

Il risultato della predizione viene poi restituito al `PredictionController`, che a sua volta si occupa di gestire la risposta HTTP da inviare al client, includendo il valore predetto nel formato appropriato.

Nome	Descrizione
Ricerca applet	L'utente IFTTT deve poter cercare una qualsiasi applet per nome.
Ricerca del canale	L'utente IFTTT deve poter cercare un servizio o canale per nome.
Ricerca applet di un canale	L'utente IFTTT deve poter cercare un'applet che coinvolge un canale specifico.
Visualizzare lista di canali	L'utente IFTTT deve poter visualizzare tutti i canali disponibili su IFTTT.
Visualizzare lista di applet	L'utente IFTTT deve poter visualizzare tutte le applet che coinvolgono un canale specifico.
Visualizzare storico predizioni	L'utente IFTTT deve poter visualizzare tutte le predizioni effettuate sulle applet.
Visualizzare dettagli applet	L'utente IFTTT deve poter visualizzare tutti i dettagli relativi a un'applet.
Effettuare predizione	L'utente IFTTT deve poter predire le applet selezionate.
Visualizzare problemi di sicurezza	L'utente IFTTT deve poter visualizzare perché e come un'applet potrebbe causare danni, e informarsi sui potenziali rischi.

Tabella 3.1: Tabella dei requisiti funzionali

CAPITOLO 4

Analisi dei Risultati

In questo capitolo analizzerò i risultati ottenuti durante la sperimentazione descritta nel Capitolo 3, rispondendo alle **domande di ricerca** (*Research Questions*) formulate nello stesso capitolo.

Ogni volta che nel corso della trattazione si faceva riferimento a una domanda di ricerca specifica, ho inserito un richiamo per identificare il tipo e il numero della domanda, rimandando a questo capitolo per una risposta più dettagliata:

- **(RQ1)** *Quali criteri devono essere considerati per determinare se un'applet è potenzialmente dannosa?* I criteri principali da valutare sono il contesto operativo dell'applet e il tipo di danno che essa può causare agli utenti. Nello specifico, i rischi si suddividono in tre categorie: **fisico, personale e sicurezza informatica**.
- **(RQ2)** *Perché è importante ampliare il dataset esistente?* L'ampliamento del dataset è fondamentale poiché il dataset attuale presenta un numero limitato di istanze e non contiene applet etichettate come innocue, rendendo l'analisi meno rappresentativa e meno efficace.
- **(RQ3)** *Qual è lo scopo principale della creazione di una web app per la valutazione delle applet?* Lo scopo principale è quello di fornire uno strumento altamente

usabile e accessibile, permettendo un utilizzo esteso da parte degli utenti di IFTTT per valutare le applet in modo semplice e immediato.

- **(RQ4)** *In che modo una web app può aiutare gli utenti a identificare e comprendere i rischi legati alle applet IFTTT?* La web app informa gli utenti sui possibili rischi attivando una determinata applet, stimando il danno potenziale in tre aree: **personale**, **fisico** e **cybersecurity**, consentendo così una comprensione chiara dei pericoli.
- **(RQ5)** *Come vengono recuperate e gestite le applet disponibili su IFTTT per la loro integrazione nel database?* Poiché IFTTT non fornisce API pubbliche per il recupero delle applet, le informazioni sono state ottenute tramite una fase di **Web Scraping**. In questa fase, si accede a una pagina web fornendo l'URL e, tramite l'analisi del DOM, vengono estratte le informazioni rilevanti.

4.1 Discussione sul dataset

Come descritto nel paragrafo 3.1, sono stati utilizzati due dataset per costruire un dataset finale, privo dei limiti e dei problemi emersi in precedenza. In pratica, questo nuovo dataset contiene le stesse istanze del dataset non etichettato, con l'aggiunta della feature `target`, dove è stata predetta la sicurezza delle applet utilizzando la tecnica della **similarità del testo**, basandosi sulle etichette del secondo dataset.

Il risultato è un dataset con le seguenti caratteristiche:

- Un insieme di 50.228 applet.
- 24 feature testuali, con l'eccezione della variabile `target`.
- Circa **più della metà** delle applet ha come `target` il valore 0, quindi sono state valutate come innocue.

Come menzionato in precedenza, i risultati ottenuti tramite la tecnica semi-supervisionata per la predizione della sicurezza delle applet sono **in linea** con quelli riportati in altre ricerche, come ad esempio in [1]. Tuttavia, ritengo che una piccola percentuale di applet etichettate come innocue possa in realtà essere dannosa, poiché

l'analisi delle **matrici di confusione** prodotte dall'addestramento dei modelli ha evidenziato che il modello tendeva a commettere più errori sull'etichetta 0.

Le etichette utilizzate sono basate sullo studio di [10], che a sua volta si riferisce a [1]. Nel paragrafo 2.2 ho citato la problematica legata all'assegnazione di etichette **non sempre precise**, in quanto non coprono tutte le casistiche analizzate. In conclusione, il dataset può essere considerato affidabile, poiché si basa sui risultati di uno studio solido e confermato, come dimostrato anche in [10], che ha reclutato un gruppo di utenti per valutare un campione di applet. Tuttavia, sarebbe opportuno rivedere le etichette per **specializzare** i tipi di rischi a cui gli utenti possono andare incontro.

4.2 Discussione sul modello di Machine Learning

Il modello di machine learning è stato addestrato utilizzando l'algoritmo SVM (Support Vector Machine), ottimizzato attraverso una selezione di **iperparametri** specifici. Di seguito, vengono riportati gli iperparametri considerati e una breve descrizione delle scelte fatte:

- **Kernel:** Il parametro `kernel` determina la funzione di kernel utilizzata dall'SVM. Le opzioni disponibili includono il kernel lineare, il kernel RBF, polinomiale e sigmoide, ciascuno con diverse proprietà in base alla separabilità e alla non-linearità dei dati.
- **C:** `C` controlla il grado di penalizzazione degli errori di classificazione. Un valore più alto di `C` penalizza maggiormente gli errori, riducendo il margine di separazione tra le classi, mentre un valore più basso consente un margine più ampio e una maggiore tolleranza agli errori, favorendo la generalizzazione.
- **Gamma:** `Gamma` regola l'influenza di ciascun esempio di addestramento sui margini del modello. Valori più alti di `gamma` restringono l'influenza, mentre valori più bassi allargano l'influenza.
- **Degree:** Questo parametro è utilizzato quando si impiega il kernel polinomiale e specifica il grado del polinomio. Maggiori valori del grado aumentano la complessità del modello.

- **Coef0:** `Coef0` rappresenta il termine indipendente nei kernel polinomiali e sigmoidali. Viene utilizzato per influenzare l'andamento del modello in combinazione con questi kernel.
- **Shrinking:** Questo parametro decide se abilitare l'euristica di restringimento per ridurre il numero di vettori di supporto e velocizzare il calcolo.
- **Probability:** `Probability` indica se abilitare la stima delle probabilità per le classi, permettendo di ottenere non solo la classe predetta ma anche la probabilità associata alla predizione.
- **Class_Weight:** `Class_Weight` assegna pesi differenti alle classi per gestire eventuali squilibri nel dataset, permettendo di dare più importanza alle classi meno rappresentate.
- **Decision_Function_Shape:** Questo parametro determina la strategia utilizzata per problemi multi-classe. L'opzione `ovo` (One-vs-One) crea classificatori binari tra tutte le coppie di classi, mentre `ovr` (One-vs-Rest) confronta ogni classe contro tutte le altre.
- **Random_State:** `Random_State` imposta il seme per la generazione casuale, garantendo che i risultati siano riproducibili.
- **Tol:** `Tol` rappresenta la tolleranza per il criterio di arresto del solutore. Un valore più basso aumenta la precisione, ma rallenta il calcolo.
- **Max_Iter:** `Max_Iter` specifica il numero massimo di iterazioni del solutore, influenzando il tempo di addestramento del modello.

Il modello è stato valutato attraverso le seguenti metriche: **accuracy**, **precision**, **recall**, **F1 score** e la **matrice di confusione**. I risultati ottenuti sono molto consistenti, con valori delle metriche non troppo variabili, a dimostrazione della robustezza del dataset e dell'efficacia dell'algoritmo scelto.

Nelle tabelle 4.1 e 4.2, sono riportati i valori esatti delle prestazioni del modello.

Nonostante le prestazioni del modello siano **molto elevate**, si osserva qualche errore di predizione in più per il **target 0**, come evidenziato dalla matrice di confusione. Inoltre, la **potenza predittiva reale** del modello non dipende esclusivamente

Metriche	Valori
Accuracy	0.9515
Precision	0.9514
Recall	0.9515
F1 Score	0.9514

Tabella 4.1: Risultati delle metriche di valutazione del modello

Confusion Matrix				
	Classe 0	Classe 1	Classe 2	Classe 3
Classe 0	1411	43	30	48
Classe 1	33	1445	13	9
Classe 2	32	10	1163	19
Classe 3	32	11	22	1903

Tabella 4.2: Matrice di confusione del modello

dalle metriche di valutazione, ma anche dalla qualità e affidabilità del dataset su cui è stato addestrato.

4.3 Risultati del Web Scraping

La fase di web scraping ha portato alla raccolta di un totale di 9070 applet e 995 servizi dalla piattaforma IFTTT. Questo risultato rappresenta una base di dati significativa per lo studio e l'analisi del funzionamento delle applet, anche se non completa rispetto al numero totale di applet disponibili. Infatti, su IFTTT sono presenti oltre 15000 applet. Tuttavia, alcune applet non sono state correttamente inserite nella raccolta finale a causa di diversi problemi riscontrati durante il processo di scraping.

Uno degli errori principali che ha influenzato la raccolta è stata l'errata assunzione che ogni applet avesse un **nome univoco**. Questo ha causato la mancata distinzione

tra alcune applet, con conseguenti duplicazioni o omissioni durante la fase di scraping. Inoltre, un altro problema rilevante è stato la confusione tra le **descrizioni delle action** e quelle delle **query**, che ha portato a una categorizzazione errata di alcune informazioni, complicando ulteriormente la corretta raccolta dei dati.

Ammetto che questa fase del progetto è stata progettata e implementata con una certa superficialità, senza tenere adeguatamente conto dell'ottimizzazione delle operazioni. In particolare, uno dei principali limiti è stato l'esecuzione di operazioni di web scraping ripetute sulle stesse pagine. Questo non solo ha reso l'intero processo meno efficiente, ma ha anche aumentato il rischio di ottenere risultati ridondanti e di sovraccaricare inutilmente i server, causando potenziali rallentamenti.

Nonostante le problematiche riscontrate, ritengo che questo strumento di scraping rappresenti comunque un'importante risorsa. Oltre a essere utile per la raccolta di dati finalizzati all'analisi della sicurezza delle applet, il dataset generato può essere sfruttato per condurre ulteriori studi e approfondimenti sul funzionamento e sulle interazioni delle applet di IFTTT. Con una base di dati di circa 9000 applet e quasi 1000 servizi, è possibile esplorare una vasta gamma di relazioni tra trigger e action, offrendo una visione d'insieme dettagliata del comportamento delle applet anche se il dataset non copre l'intera popolazione di applet disponibili.

Un altro punto di forza dello scraping eseguito è la raccolta dei **link dettagliati** relativi alle action e ai trigger, che coinvolgono specifici canali di IFTTT. Questi link possono essere estremamente utili per sviluppare future versioni dell'applicazione o per ampliare l'analisi in maniera più granulare, consentendo di studiare in profondità le interazioni tra applet e i servizi collegati. Questo permette di sfruttare i dati raccolti non solo per la sicurezza delle applet, ma anche per l'analisi dei canali e delle loro peculiarità, creando opportunità per progetti futuri.

4.4 Valutazione di IFTTT Security Extension

Al termine dell'implementazione della web app, è stato necessario effettuare una **valutazione** dei requisiti e degli obiettivi definiti durante la fase di analisi. In questa sezione, esaminerò principalmente due aspetti fondamentali:

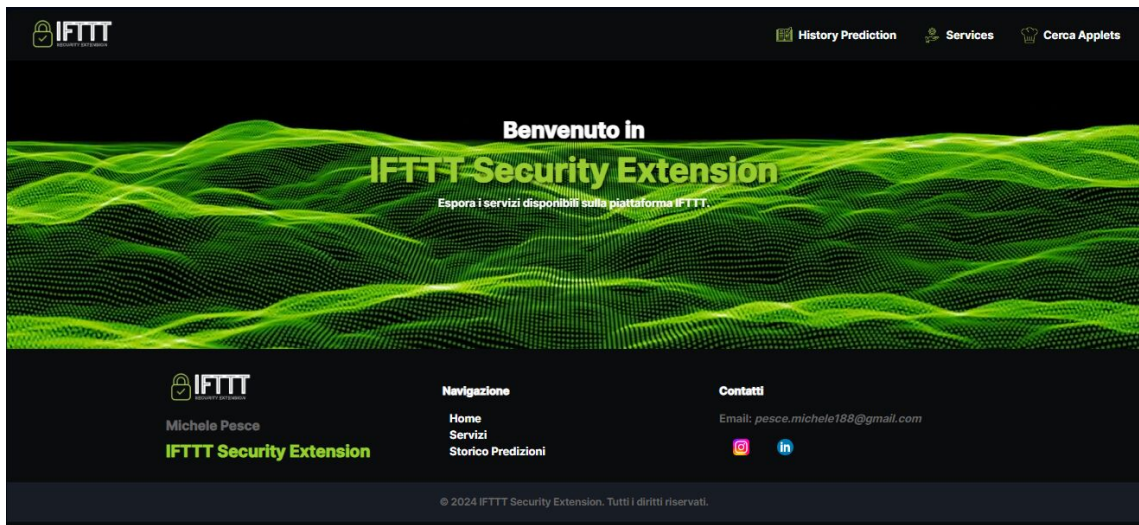


Figura 4.1: Homepage di IFTTT Security Extension

- Se sono stati rispettati i **requisiti funzionali**.
- Se i **requisiti non funzionali** sono stati adeguatamente soddisfatti.

Per una valutazione più accurata, ho deciso di sottoporre l'applicazione a un test con 10 utenti variegati: da quelli **meno esperti** a quelli **più esperti** in ambito tecnologico, al fine di ottenere un'ampia gamma di opinioni.

4.4.1 Valutazione delle funzionalità

Nella tabella 3.1 sono state descritte le funzionalità che l'applicazione doveva implementare. A implementazione conclusa, posso confermare che **tutte le funzionalità sono state progettate e implementate correttamente**. Oltre a una mia valutazione personale sulla **robustezza** dell'applicazione, ho fatto testare l'applicazione agli utenti, fornendo loro istruzioni su cosa fare **senza guidarli nella navigazione**.

Posso confermare che **tutti** i tester sono riusciti a completare tutte le funzionalità sopra descritte, senza riscontrare errori durante l'esecuzione. Gli utenti più esperti

hanno tentato di navigare digitando manualmente degli URL casuali, nel tentativo di ottenere risultati imprevisti, ma è subito apparso un *alert* personalizzato (un componente React creato da zero) che li ha informati dell'inesistenza della pagina, reindirizzandoli alla *Homepage*.

Altri utenti hanno cercato di interagire con elementi di background quando apparivano pop-up, ma finché questi non venivano chiusi, non era possibile eseguire operazioni sulle pagine sottostanti.

Alcuni tester, specialmente quelli meno esperti, hanno mostrato interesse per la problematica della sicurezza in IFTTT. Infatti, al termine della predizione delle applet, appare una pagina che informa l'utente sui potenziali rischi, accompagnata da un esempio illustrativo (come mostrato in figura 4.2). L'obiettivo principale era sensibilizzare gli utenti di IFTTT sui problemi di sicurezza, e dato che questo interesse è stato suscitato anche in utenti non utilizzatori di IFTTT, è altamente probabile che lo stesso effetto si verifichi con gli utilizzatori abituali di applet.

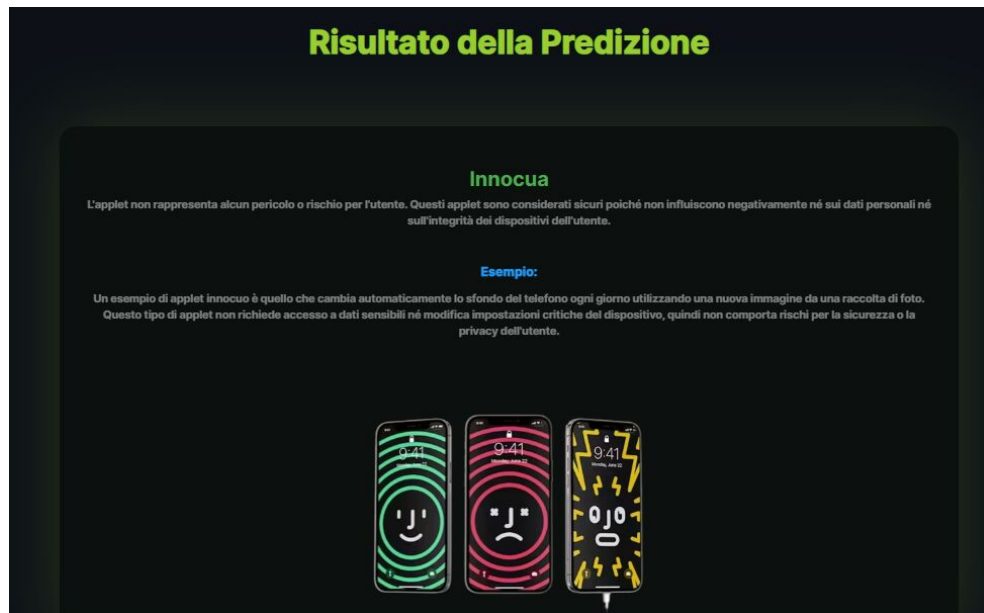


Figura 4.2: Predizione di IFTTT Security Extension

4.4.2 Valutazione dell'usabilità e dell'efficienza

Un aspetto critico su cui mi sono concentrato riguarda i **requisiti non funzionali**. La manutenibilità era essenziale per gestire correttamente il grande flusso di dati proveniente da IFTTT. Posso confermare che, dopo aver apportato modifiche significative a un modulo indipendente, gli altri moduli non hanno subito alterazioni o malfunzionamenti.

Dal punto di vista dell'**usabilità** e dell'**efficienza**, ho richiesto una valutazione critica dagli utenti già menzionati. Anche gli utenti più esperti hanno giudicato la scelta di separare le operazioni di frontend e backend come **completamente corretta**. Hanno infatti riscontrato che la selezione delle applet e dei relativi dettagli era molto rapida e reattiva. Alcuni di loro hanno commentato di non percepire la **separazione** tra backend e frontend, grazie alla fluida interazione con le chiamate API. Inoltre, hanno apprezzato la fluidità delle interazioni tra le pagine.

L'unico punto di critica riguarda la **predizione**. Alcuni utenti hanno segnalato che, sebbene l'algoritmo fosse leggermente più performante, questo avveniva a discapito della velocità. In alcuni casi, la predizione risultava lenta, specialmente nelle prime applet testate.

Dal punto di vista dell'**usabilità**, ho stilato una lista di pagine da navigare per eseguire le operazioni e verificato che tutti gli utenti seguissero il percorso corretto. Nella maggior parte dei casi, la procedura è stata rispettata, ma gli utenti meno esperti hanno incontrato difficoltà nel **ricercare l'applet tramite la barra di ricerca**, poiché questa funzione è posizionata nella navbar (come illustrato in figura 4.3).

Le difficoltà riscontrate sono dovute principalmente a due fattori:

1. La funzione di ricerca non riceveva abbastanza rilievo essendo posizionata nell'header.
2. L'icona del cappello da chef non era intuitiva e non rappresentava chiaramente la funzione di ricerca.

Infine, riguardo alla godibilità dell'interazione con l'applicazione, ho ricevuto solo feedback positivi. Molti utenti hanno apprezzato la scelta dei colori e l'estetica generale dell'applicazione, mentre altri hanno elogiato la fluidità delle interazioni.

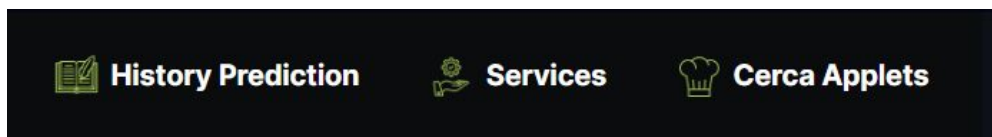


Figura 4.3: Navbar di IFTTT Security Extension

CAPITOLO 5

Conclusioni

Come già accennato nell'introduzione, questa ricerca ha sviluppato una serie di strumenti utili per esplorare in modo più approfondito i problemi legati alla sicurezza delle applet:

- È stato creato un **dataset** affidabile, con una serie di applet associate a una **etichetta di sicurezza**. Come discusso nel paragrafo 4.1, il bilanciamento dei target risulta conforme ai risultati ottenuti in precedenti ricerche.
- Il dataset ha permesso la creazione di un **classificatore** con parametri prestazionali molto elevati e stabili (4.2). Inoltre, è stata implementata una pipeline per gestire la sovrabbondanza di feature generate durante la vettorizzazione. Infine, è stato sviluppato un metodo efficace e conveniente per l'**integrazione del modello** in ambiente Java.
- È stato creato un database facilmente integrabile in qualsiasi applicazione, contenente riferimenti a tutte le applet, ai loro creatori e ai servizi disponibili su IFTTT nel 2024. Questo strumento consente uno studio più approfondito delle **applet moderne**, con un focus particolare sulle **percezioni** e **preoccupazioni** degli utenti IFTTT.

- Infine, è stata realizzata una **web app** che, oltre a predire se un'applet selezionata dall'utente sia dannosa o meno, mira a **sensibilizzare** l'utente sui potenziali rischi legati all'attivazione di applet dannose.

Per ottenere stime ancora più accurate, sarebbe stato utile approfondire l'analisi della **soglia di similarità** (3.1.3), in modo da ridurre gli errori nella predizione di **applet innocue** (con target 0).

Inoltre, come discusso in 3.4, a causa della mancanza di un calcolatore ad alte prestazioni, l'integrazione dei **trigger** e delle **action** associati ai canali/servizi presenti su IFTTT è stata rimandata a future versioni. Ciò richiederà non solo un ulteriore studio sui marcatori utili per l'estrazione dei dati durante il web scraping, ma anche una **modifica** del database, poiché sarà necessario distinguere tra trigger e action separati dalle applet già esistenti.

Questo lavoro potrebbe portare allo sviluppo di una **nuova versione** di IFTTT Security Extension, con nuove funzionalità dedicate alla creazione di applet personalizzate dall'utente, a partire da trigger e action selezionati. Tuttavia, la costruzione di applet richiede un'attenta valutazione della loro **logica**, al fine di determinare se un'applet creata abbia effettivamente senso. A tal fine, sarà necessario **filtrare** le action rilevanti da quelle inutili, una volta scelto un trigger.

5.1 Sviluppi futuri

Il campo della sicurezza legata alle applet è vasto e complesso. Esistono applet con problemi di sicurezza intrinseca, ossia potenzialmente dannose per loro natura, così come applet con **code filters** manomessi per esfiltrare informazioni private degli utenti.

Relativamente a quest'ultimo problema, sarebbe utile condurre una ricerca mirata. Spesso l'attenzione si concentra sulle applet intrinsecamente insicure, ma molte di esse vengono create da sviluppatori esterni al servizio fornitore. In questo contesto, sarebbe interessante:

- Condurre uno studio su quante applet siano state create da terzi e quanti utenti utilizzino regolarmente queste applet.

- Analizzare e creare **code filters** volutamente malevoli per comprendere meglio la gravità del problema e sviluppare una **soluzione** efficace per gestirlo.
- Realizzare un modello di **intelligenza artificiale** capace di prevedere quali applet potrebbero essere soggette ad attacchi da parte di creatori malintenzionati, nonché la **gravità** dei danni che potrebbero arrecare agli utenti IFTTT. A tal proposito, la ricerca [7] ha sviluppato due dataset di **actions** e **triggers** etichettati secondo criteri specifici. Questi dataset, essendo pubblici, potrebbero essere utilizzati per costruire il modello e predire applet potenzialmente dannose.

Un ulteriore sviluppo futuro interessante riguarda la **sensibilizzazione** degli utenti di IFTTT. Sarebbe utile analizzare come, nel corso degli anni, sia cambiata la consapevolezza dei rischi legati all'uso delle applet. Questo potrebbe includere uno studio con un campione eterogeneo di utenti, coinvolgendoli in questionari e test per valutare come è evoluta la loro percezione e quali siano le preoccupazioni attuali riguardanti l'utilizzo delle applicazioni **trigger-action**.

Sulla base di queste considerazioni, un ulteriore filone di ricerca potrebbe concentrarsi sui **metodi migliori per sensibilizzare gli utenti**. Nella mia ricerca ho ipotizzato che una **web app** altamente usabile rappresenti la scelta ottimale per diversi motivi, ma uno studio più approfondito potrebbe fornire dati più solidi. Inoltre, molti utenti di IFTTT utilizzano prevalentemente l'**applicazione mobile**, e potrebbero non essere abituati a interagire con le applet sul web. Con l'aumento dell'integrazione della **realtà aumentata** e della **visione artificiale**, si potrebbe anche considerare lo sviluppo di un'applicazione immersiva, che utilizzi visori per rafforzare i concetti di sicurezza che vogliamo esporre.

Bibliografia

- [1] L. B. A. D. L. J. Milijana Surbatovich, Jassim Aljuraidan, "Some recipes can do more than spoil your appetite: Analyzing the security and privacy risks of ifttt recipes," *Carnegie Mellon University*, 2017. [Online]. Available: https://www.researchgate.net/publication/313889275_Some_Recipes_Can_Do_More_Than_Spoil_Your_Appetite_Analyzing_the_Security_and_Privacy_Risks_of_IFTTT_Recipes (Citato alle pagine iv, 3, 16, 17, 18, 27, 35, 60 e 61)
- [2] V. Kanade, "What is ifttt (if this then that)? meaning, working, and alternatives," *Spiceworks*, 2023. [Online]. Available: <https://www.spiceworks.com/tech/tech-general/articles/what-is-ifttt/> (Citato a pagina 3)
- [3] P. Software, "Spring boot," 2024, accessed: 2024-09-04. [Online]. Available: <https://spring.io/projects/spring-boot> (Citato a pagina 4)
- [4] Wikipedia, "Pmml," *wikipedia*, 2016. [Online]. Available: https://it.wikipedia.org/wiki/Predictive_Model_Markup_Language (Citato alle pagine 4 e 43)
- [5] —, "Mvc," *wikipedia*, 2016. [Online]. Available: <https://it.wikipedia.org/wiki/Model-view-controller> (Citato a pagina 4)
- [6] I. Facebook, "React," 2024, accessed: 2024-09-04. [Online]. Available: <https://reactjs.org> (Citato a pagina 4)

- [7] A. S. Iulia Bastys, Musard Balliu, "If this then what? controlling flows in iot apps," *Chalmers University of Technology, KTH Royal Institute of Technology*, 2018. [Online]. Available: <https://dl.acm.org/doi/10.1145/3243734.3243841> (Citato alle pagine 13, 14 e 71)
- [8] JSflow, "Jsflow," 2024, accessed: 2024-09-04. [Online]. Available: <https://www.jsflow.net> (Citato a pagina 16)
- [9] V. D. A. E. Bernardo Breve, Gaetano Cimino, "User perception of risks associated with ifttt applets: A preliminary user study," *Dipartimento di Informatica, Università degli Studi di Salerno, Fisciano, Italia*, 2023. [Online]. Available: <https://ceurws.org/Vol3488/paper07.pdf> (Citato a pagina 19)
- [10] B. Breve, G. Cimino, and V. Deufemia, "Identifying security and privacy violation rules in trigger-action iot platforms with nlp models," *IEEE Internet of Things Journal*, 2022. (Citato alle pagine 20, 26, 27, 28, 31 e 61)
- [11] M. Saeid, M. Calvert, A. W. Au, A. Sarma, and R. B. Bobba, "If this context then that concern: Exploring users' concerns with ifttt applets," ResearchGate, 2021. [Online]. Available: <https://arxiv.org/abs/2012.1251> (Citato alle pagine 21 e 22)
- [12] M. S. C. Cobb, "How risky are real users' ifttt applets?" *Carnegie Mellon University*, 2020. (Citato alle pagine 23 e 35)

Ringraziamenti

Un ringraziamento speciale va alla **mia famiglia**, che con amore e costanza mi ha sempre sostenuto, credendo in me e nelle mie scelte, e permettendomi di seguire i miei sogni e le mie aspirazioni accademiche. Senza il loro affetto, nulla di tutto questo sarebbe stato possibile.

Un sincero e profondo grazie al Prof. **Fabio Palomba**, il mio relatore, per la sua pazienza, la sua disponibilità e per i preziosi consigli che mi hanno guidato passo dopo passo nella stesura della tesi. La sua dedizione è stata per me fonte di ispirazione.

Un pensiero speciale va a **Carmine, Salvatore e Francesco**, con i quali ho condiviso non solo il percorso universitario, ma anche momenti di reciproco supporto, soprattutto nei progetti più impegnativi. La nostra amicizia è stata una vera risorsa.

Grazie di cuore a **Domenico**, mio caro amico ed ex coinquilino, per le infinite chiacchierate serali che mi hanno regalato momenti di leggerezza, aiutandomi a rilassarmi dopo lunghe giornate di studio intenso.

Un ringraziamento di cuore va a **Rosa**, la mia preziosa amica. La sua presenza costante nei momenti difficili, i suoi consigli saggi e il suo sostegno incondizionato mi hanno aiutato a superare le sfide universitarie e di vita. Sono profondamente grato per tutto l'aiuto che mi ha dato.

Infine, un riconoscimento va a **me stesso**, per la forza, la determinazione e la resilienza che mi hanno permesso di superare ogni ostacolo lungo il cammino. Questo

traguardo è frutto anche della mia capacità di non arrendermi mai.